# NORTHSTAR: INSIGHTS FOR EVERY DEVELOPMENT JOURNEY

**An Engineering Intelligence Platform**

## *Report*

*Submitted in partial fulfillment of the requirements*
*for the award of the degree of*

**Bachelor of Technology**
*in*
**Computer Science and Engineering - Artificial Intelligence**

*by*

**Raunak Pandey (2301921520147)**

*under the supervision of*

Ms. Priyanka Chandra
Assistant Professor
Department of Computer Science and Engineering - AI

**G. L. BAJAJ INSTITUTE OF TECHNOLOGY & MANAGEMENT**
**GREATER NOIDA, UTTAR PRADESH - 201306**

*Affiliated To*

**DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY**
**LUCKNOW, UTTAR PRADESH - 226031**

2026 – 2027

# Declaration

I declare that this written submission represents my ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all academic principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will cause disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

**RAUNAK PANDEY**
Roll No.: 2301921520147

Dated:

# Certificate

This is to certify that the dissertation work entitled **"NorthStar: Insights for Every Development Journey"** is a bonafide record of the work carried out by **"Raunak Pandey (2301921520147)"**, submitted to the department of **"Computer Science and Engineering - AI"**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering - AI** at the G L Bajaj Institute of Technology & Management during the academic year 2026-2027.

**Supervisor**

Ms. Priyanka Chandra
Assistant Professor
Department of CSE - AI
G. L. Bajaj Institute of
Technology & Management

**Head of the Department**

Dr. Anurag Mishra
HOD of CSE-AI
Department of CSE - AI
G. L. Bajaj Institute of
Technology & Management

# Acknowledgment

I express my sincere gratitude and indebtedness to our respected guide, Ms. Priyanka Chandra, Assistant Professor, Department of Computer Science & Engineering - AI, G. L. Bajaj; thank you for providing us with much-needed support and guidance to undertake this research work. His timely help and invaluable suggestions paved the way for the research to be of respectable quality and insight. To all the faculty and office staff of the Department of Computer Science & Engineering - AI, G. L. Bajaj, I extend my deepest gratitude for their unrelenting support throughout the course of this research and degree study.

I also acknowledge the open-source communities and documentation teams behind Go, PostgreSQL, Docker and GitHub, whose tools and technical resources made this project possible, and everyone who offered suggestions and feedback during its planning and development.

**RAUNAK PANDEY**
Roll No.: 2301921520147

# Abstract

Modern software teams use many specialised tools to manage source code, code reviews, continuous integration, issue tracking, deployments and infrastructure monitoring. Each tool generates valuable engineering data, but that data is distributed across independent systems in different formats. As a result, engineering teams and technical leaders find it difficult to form a unified, evidence-based view of how software is being built, tested, deployed and improved, and everybody in a team tends to answer the question “how are we doing?” differently.

This project presents NorthStar, an Engineering Intelligence Platform whose tagline is “Insights for Every Development Journey”. NorthStar is designed to collect engineering activity from the tools that teams already use, normalise it into consistent records, calculate clearly defined metrics and present the results to different engineering roles through a REST API, dashboards and reports. The conceptual design rests on six pillars: source integration, an analytics engine, a rule-based intelligence layer, dashboards, reports and goals. The product is planned in four versions: GitHub analytics, CI/CD and deployment analytics, DORA metrics (deployment frequency, lead time for changes, change failure rate and recovery time) and team intelligence.

The first release is deliberately scoped as a modular Go application backed by PostgreSQL and packaged with Docker and Docker Compose, instead of a set of microservices, with message brokers, Kubernetes and AI-based recommendations explicitly deferred. The report documents the requirements, architecture, GitHub integration design, data model, metric definitions with their limitations, API design, configuration and security design, and a testing strategy.

At the time of submission the application foundation has been implemented and verified: the Go application starts, loads configuration and logs; the container and PostgreSQL environment is configured; and a secret-leak incident was identified, remediated and converted into engineering rules. The GitHub integration is the current milestone, while the collector, persistence, analytics, API and dashboard are planned. The report states implemented and planned work separately and supports the plan with a worked metric example using clearly labelled hypothetical data.

**Keywords:** Engineering Intelligence, DevOps Analytics, DORA Metrics, Software Delivery Performance, Go, GitHub API, PostgreSQL, Docker, CI/CD.

# Contents

**Declaration**.....i
**Certificate**.....ii
**Acknowledgment**.....iii
**Abstract**.....iv
**List of Figures**.....viii
**List of Tables**.....ix
**1 Introduction**.....1
1.1 Background.....1
1.1.1 Software Delivery and Toolchain Fragmentation.....1
1.1.2 Engineering Metrics and the DORA Framework.....2
1.2 Role in Real Life.....3
1.3 Engineering Intelligence and the Developer Journey.....3
1.4 Evolution of the Idea.....4
1.5 Problem Statement.....5
1.6 Motivation.....5
1.7 Brief about Proposed Approach.....6
1.8 Objectives.....6
1.9 Contributions.....7
1.10 Organisation of the Report.....8
**2 Literature Review**.....9
2.1 Introduction.....9
2.2 Research on Software Delivery Performance.....9
2.3 Measuring Developer Productivity.....10
2.4 Continuous Integration and Continuous Delivery.....11
2.5 Existing Engineering Analytics Tools.....11
2.6 Architectural Styles.....12
2.6.1 Monolith, Modular Monolith and Microservices.....12
2.6.2 Event-Oriented Architecture.....13
2.6.3 REST.....13
2.7 The Go Programming Language for Infrastructure Software.....13
2.8 Persistence and Data Ingestion.....14
2.9 Containerisation and Configuration.....15
2.10 Observability.....15
2.11 Research Gap and Positioning.....15
2.12 Summary.....16
**3 Proposed Model/Framework**.....17
3.1 Design Philosophy.....17
3.2 The Six Pillars of NorthStar.....17
3.3 Users, Dashboards and Reports.....18
3.4 Requirements.....19
3.4.1 Functional Requirements.....19

3.4.2 Non-Functional Requirements ..... 19
3.5 System Architecture ..... 20
3.5.1 Long-term Architecture ..... 20
3.5.2 Version 1 Architecture ..... 21
3.5.3 Mapping the Original Four Services to Modules ..... 21
3.6 Repository and Project Structure ..... 22
3.6.1 Rationale for Key Structural Decisions ..... 23
3.7 Technology Stack ..... 23
3.7.1 Why Go? ..... 24
3.7.2 Why PostgreSQL? ..... 24
3.7.3 Why Docker? ..... 25
3.8 GitHub Integration Design ..... 25
3.8.1 The API Client ..... 25
3.8.2 Models and Resource Operations ..... 26
3.9 Data Collection Strategy ..... 27
3.9.1 Polling and Scheduling ..... 27
3.9.2 Pagination, Rate Limits and Retries ..... 27
3.9.3 Incremental Synchronisation and Idempotency ..... 28
3.9.4 Data Normalisation ..... 28
3.10 Data Model ..... 28
3.10.1 Integrity Considerations ..... 29
3.10.2 Database Connection URL ..... 29
3.11 Metric Definitions ..... 30
3.11.1 Version 1: Engineering Activity Metrics ..... 30
3.11.2 Version 2: Delivery Metrics ..... 30
3.11.3 Version 3: DORA Metrics ..... 31
3.11.4 Version 4: Team Intelligence ..... 31
3.12 Analytics Engine and Intelligence Layer ..... 31
3.13 REST API Design ..... 32
3.14 Application Startup and Composition Root ..... 33
3.15 End-to-End Workflow ..... 33
3.16 Configuration and Secrets Management ..... 34
3.17 Containerisation Design ..... 35
3.18 Security Design ..... 35
3.18.1 Credential Management ..... 35
3.18.2 API and Application Security ..... 35
3.19 Concurrency Design ..... 36
3.20 Testing and Documentation Framework ..... 37
3.21 Product Roadmap ..... 37
3.21.1 What Was Deliberately Excluded from Version 1 ..... 38
3.21.2 Architectural Traps to Avoid ..... 38
**4 Experimentation & Results Discussion** ..... 39
4.1 Introduction and Methodology ..... 39

4.2 Experimental Environment....39
4.3 Verification 1: Application Foundation....40
4.4 Verification 2: Configuration Management....40
4.5 Verification 3: Container Foundation....41
4.5.1 Database Host Name inside Docker Compose....41
4.5.2 Volume Persistence....41
4.6 Verification 4: Secret-Leak Incident and Remediation....42
4.7 Verification 5: Repository Audit....43
4.8 Progress Summary....43
4.9 Evaluation Plan for Version 1....44
4.9.1 Acceptance Criteria for GitHub Version 1....45
4.9.2 Validating Metric Correctness....45
4.10 Worked Example of Metric Computation (Illustrative Data)....45
4.11 Discussion....46
4.11.1 What the Results Show....46
4.11.2 Learning Outcomes....46
4.11.3 Design Decisions Validated....47
4.11.4 Threats to Validity and Limitations....47
**5 Conclusion and Future Work**....49
5.1 Conclusion....49
5.2 Future Work....50
5.2.1 Completing Version 1....50
5.2.2 Subsequent Versions....50
5.2.3 Platform Enhancements....51
5.2.4 Engineering and Community Practice....51
5.3 Final Remarks....51
**References**....52
**Appendix A: Glossary**....54
**Appendix B: Abbreviations**....55
**Appendix C: Illustrative API Response**....56
**Appendix D: Project Planning Notes Summary**....57

# List of Figures

Figure 3.1: Long-term (target) architecture of NorthStar.....20
Figure 3.2: NorthStar version 1 modular architecture.....21
Figure 3.3: Proposed logical data model.....28
Figure 3.4: Planned application startup sequence.....33
Figure 3.5: End-to-end data workflow.....34
Figure 3.6: Simplified model of the Go scheduler (G-M-P).....36
Figure 3.7: NorthStar product roadmap.....37

# List of Tables

Table 1.1: Intended users of NorthStar and their information needs....3
Table 1.2: Evolution of the product vision....4
Table 2.1: DORA four key metrics and the data required to compute them....10
Table 2.2: Characteristics of existing approaches and the NorthStar position....12
Table 3.1: The six pillars of NorthStar....17
Table 3.2: Dashboards and their principal content....18
Table 3.3: Functional requirements for the initial release....19
Table 3.4: Non-functional requirements....19
Table 3.5: Responsibilities of the version 1 components....21
Table 3.6: Original services and corresponding version 1 modules....22
Table 3.7: Technology stack, purpose and status....23
Table 3.8: Design decisions for the GitHub client....25
Table 3.9: GitHub endpoints used by the version 1 collector....27
Table 3.10: Proposed data entities....29
Table 3.11: Version 1 activity metrics....30
Table 3.12: Planned formulas for the DORA metrics....31
Table 3.13: Planned REST endpoints for version 1....32
Table 3.14: Roadmap by version....37
Table 4.1: Development and experimentation environment....39
Table 4.2: Configuration behaviours and verification status....40
Table 4.3: Effect of Compose shutdown commands on data....41
Table 4.4: Timeline and remediation of the token exposure....42
Table 4.5: Top-level contents of the repository at the time of review....43
Table 4.6: Development progress at the time of submission....43
Table 4.7: Test levels and planned validation....44
Table 4.8: Worked DORA example with hypothetical data....46
Table 5.1: Implementation roadmap from the current position....50
Table A.1: Glossary of terms....54
Table B.1: Abbreviations used in this report....55

# Chapter 1
# Introduction

## 1.1 Background

Software has become the operating system of modern organisations. Banks, hospitals, retailers, logistics companies and governments all depend on software that must be changed continuously while it remains available to users. The ability to change software quickly and safely is therefore no longer a purely technical concern; it is a business capability. Organisations that can release small, well-tested changes frequently are able to respond to customer feedback, fix defects and adopt new technology faster than organisations that release rarely and in large batches [1].

To support this continuous change, engineering teams rely on a chain of specialised tools. Source-code platforms such as GitHub and GitLab store the code and manage collaboration through pull requests. Continuous-integration systems such as Jenkins and GitHub Actions compile the code and run automated tests. Issue trackers such as Jira organise planned work. Container platforms such as Docker and orchestration platforms such as Kubernetes package and run the resulting applications, while monitoring stacks such as Prometheus and Grafana observe them in production [13], [22], [23], [24], [29], [30]. Every one of these tools produces useful data, yet each tool stores that data in its own system, in its own format, and presents it through its own interface.

NorthStar, the subject of this report, is an Engineering Intelligence Platform designed to address the problem that arises from this arrangement. Its tagline, “Insights for Every Development Journey”, captures the central idea: every engineering activity leaves a footprint, and if those footprints can be collected and connected, they describe the complete journey of a change from the moment a developer writes the first line of code to the moment the change is running in production and has either succeeded or required recovery.

### 1.1.1 Software Delivery and Toolchain Fragmentation

A typical change passes through many stages. A developer commits code, opens a pull request, waits for review, receives comments, updates the change, merges it, and then waits for a pipeline to build and test the merged code. If the pipeline succeeds, the artefact is deployed to a staging or production environment. If the deployment misbehaves, an incident is raised and the team works to restore service. Each of these stages is recorded by a different system.

The consequence is fragmentation. A team lead can open GitHub and see how many commits were pushed this week, but that number alone says very little about whether the team is

delivering value. The more important questions span several tools: how long does a change take to reach production, how long do pull requests wait for review, how often do deployments fail, how quickly does the team recover when they do, and is the overall trend improving or degrading? Answering any of these questions requires joining data from at least two or three systems. In practice this is done manually, through spreadsheets and ad-hoc scripts, which is slow, error-prone and rarely repeated consistently.

The first observation that led to NorthStar was therefore simple. When a manager asks “how are we doing?”, everybody in the team gives a different answer, because everybody is looking at a different tool. NorthStar is intended to give one consistent, data-backed answer by telling the story the data tells together.

### 1.1.2 Engineering Metrics and the DORA Framework

Measuring software delivery has a long and contested history. Early attempts counted lines of code or the number of commits, measures that are easy to collect but that correlate poorly with value and are easy to manipulate. A significant advance came from the DevOps Research and Assessment (DORA) programme, whose multi-year research, summarised in the book Accelerate and in annual State of DevOps reports, identified a small set of measures that predict software delivery performance and organisational outcomes [1], [2].

These measures are commonly known as the four key metrics. Deployment frequency describes how often an organisation successfully releases to production. Lead time for changes describes how long it takes a committed change to reach production. Change failure rate describes the proportion of deployments that cause a failure requiring remediation. Failed-deployment recovery time, historically called mean time to restore, describes how long it takes to recover from such a failure. The first two measure throughput; the last two measure stability. The DORA research argues that high-performing teams do well on both, which contradicts the common assumption that speed and stability must be traded against each other [1], [2].

Computing these metrics is conceptually straightforward but practically difficult, because the underlying timestamps live in different tools: the commit time in the source-code platform, the build and deployment times in the CI/CD system, and the incident and recovery times in an operations tool. NorthStar is designed to be the layer that gathers these timestamps and computes the metrics in a consistent, explainable way.

## 1.2 Role in Real Life

Engineering intelligence is relevant to several roles in a software organisation, and each of them asks different questions of the same underlying data. During the planning of NorthStar these roles were identified as the primary users of the system, and the platform's dashboards are organised around them.

**Table 1.1: Intended users of NorthStar and their information needs**

| User | What they need from NorthStar |
|---|---|
| CTO | Overall engineering delivery visibility; trends over quarters; confidence that delivery is healthy. |
| Engineering manager | Team progress, delivery bottlenecks, comparison of teams and trends over sprints. |
| Project manager | Whether sprint and delivery commitments are on track; summaries for stakeholders. |
| DevOps engineer | Build, deployment and pipeline visibility; failing pipelines; deployment slowdowns. |
| Team lead | Pull requests, review times, development activity and delivery progress of the team. |
| Platform engineer | Visibility across engineering tools and workflows; integration health. |

In real organisations, the lack of such a view has concrete costs. A manager preparing a quarterly review may spend hours exporting data from several tools. A DevOps engineer may discover that deployments have been slowing down only after developers complain. A CTO may only learn that delivery has degraded when a major release slips. A platform that continuously collects and analyses the data surfaces such trends earlier, when they are cheaper to fix.

It is equally important to state what NorthStar is not. It is not intended to replace GitHub, Jenkins, Jira, Grafana or any other tool. Those tools remain the systems of record. NorthStar's role is to consume their data, never to create it, and to present a higher-level view that none of them can offer individually. It is also not intended to rank developers. Metrics such as commit counts describe activity, not productivity, and the platform is designed to communicate what each metric means and what its limitations are [3], [19], [20].

## 1.3 Engineering Intelligence and the Developer Journey

The term "engineering intelligence" is used in this report to describe the practice of collecting engineering activity data, relating it across tools, and turning it into insight about how software is built, tested, deployed and improved. The vision statement written for NorthStar expresses this succinctly: NorthStar helps an engineering organisation understand how software is built, tested, deployed and improved by transforming deployment data into meaningful insights.

The concept of a “development journey” is central. A pull request is not an isolated object; it is opened, reviewed, approved, merged, built, tested, deployed, and eventually either runs without trouble or contributes to an incident. Each step is an event with a timestamp. When these events are linked by identifiers such as commit hashes and pull-request numbers, they form a chain that can be analysed. The journey that NorthStar aims to model is shown below.

```
Commit -> Pull Request -> Review -> Merge -> CI Build -> Test
                                                          |
Recovery <- Incident <- Production <- Deployment <---------+
```

*Figure-style listing: the engineering journey NorthStar connects*

Event-oriented thinking also shapes the long-term architecture. Each stage can be represented as an event, for example pull_request.opened, pull_request.merged, build.completed or deployment.completed. An analytics engine that understands the timestamps and relationships between these events can compute lead time, review time, build success rate and recovery time without being tied to the response format of any one tool [18].

## 1.4 Evolution of the Idea

NorthStar did not begin as an engineering intelligence platform. The first concept, recorded under the working title “DevOps Analytics”, was a relatively modest project to show GitHub statistics and build history. As the idea was explored, it became clear that a team’s performance cannot be understood by looking at GitHub alone. The same realisation, repeated for build systems, issue trackers and deployment platforms, widened the idea from a statistics dashboard into a platform that connects different tools.

**Table 1.2: Evolution of the product vision**

| Stage | Name | Description |
|---|---|---|
| Original idea | DevOps Analytics | GitHub statistics, build history and basic development metrics. |
| Expanded vision | Engineering Intelligence Platform | Connect different engineering tools and consolidate their data. |
| Long-term direction | Engineering Journey Intelligence | Understand the complete journey from code creation to delivery, deployment and operational performance. |

The product name and tagline were fixed at this point. The name NorthStar is meant to communicate direction: helping engineering teams understand where they are, how they are progressing and where they should improve. The tagline, “Insights for Every Development Journey”, reflects the intention that every team, regardless of its toolchain, should be able to obtain insight.

## 1.5 Problem Statement

Engineering data is distributed across independent platforms. Source-control systems, CI/CD tools, issue trackers, deployment platforms and monitoring stacks each hold part of the story, and each exposes that part through its own data model and interface. Because no single place correlates these records, teams and engineering leaders cannot easily answer cross-tool questions such as how long a change takes to reach production, how long pull requests remain open, how frequently deployments fail, how quickly the team recovers, whether delivery speed is improving or declining, and where bottlenecks are appearing in the workflow.

The problem addressed by this project can therefore be stated formally as follows: to design and develop a modular platform that connects to the tools a software team already uses, collects and normalises their engineering activity data, computes clearly defined delivery and activity metrics from that data, and presents the results to different engineering roles through an API and dashboards, while being transparent about what each metric means and what it does not.

## 1.6 Motivation

The motivation for the project has three strands: a practical need, a technical opportunity and a learning objective.

The practical need is the fragmentation described above. Evidence-based engineering management requires data, and the data exists, but it is difficult to assemble. Commercial platforms for engineering analytics exist, but they are generally proprietary, priced per developer, and designed around their vendors' assumptions. A transparent, extensible, self-hostable platform whose metric definitions are visible and modifiable is a worthwhile alternative, and a good vehicle for studying how such systems are built.

The technical opportunity lies in the maturity of the underlying technologies. Go offers a statically typed, compiled language with first-class concurrency that is well suited to network-facing services and API clients [7], [8]. PostgreSQL is a mature relational database with strong transactional guarantees [12]. Docker provides repeatable environments [13], [14]. GitHub exposes a comprehensive REST API [11]. Together these make it feasible for a small team to build a genuinely useful platform.

The learning objective is equally important. NorthStar was conceived as a practical project through which to learn how infrastructure-oriented software is designed: HTTP clients and authentication, pagination and rate limiting, concurrency, persistence, containerisation, configuration and secret management, testing, and the software-engineering discipline of keeping early versions small. These are the skills demanded of DevOps, cloud, platform and backend engineers, and building a real system is the most reliable way to acquire them.

## 1.7 Brief about Proposed Approach

NorthStar is proposed as an engineering data collection, processing, analytics and visualisation platform. At a high level it follows a six-step conceptual pipeline: external engineering systems provide data; integrations and collectors authenticate and fetch that data; a processing layer validates and normalises it; an analytics engine calculates metrics; a storage layer persists both raw records and calculated results; and an API and presentation layer expose the results through dashboards, reports and insights.

Three decisions define the approach. First, NorthStar consumes data and never creates it; the external tools remain authoritative. Second, collection and calculation are separated: the collector only collects, the analytics engine only calculates, and the dashboard only asks the API for already-calculated metrics. Third, the first release is deliberately small. Version 1 concentrates on GitHub and is built as a modular Go application rather than as a set of microservices, with additional integrations, full DORA analytics and team intelligence deferred to later versions [17].

The platform is delivered in four planned versions. Version 1 (Engineering Activity) covers GitHub integration, commits, pull requests, contributors and repository insights. Version 2 (Delivery Activity) adds CI/CD builds, deployment history, build success and failure, and pipeline trends. Version 3 (DORA Intelligence) computes deployment frequency, lead time for changes, change failure rate and recovery time. Version 4 (Team Intelligence) adds team comparison, an engineering health score, sprint analytics, an executive dashboard and historical trends.

## 1.8 Objectives

The primary objective of the project is to design and develop a modular Engineering Intelligence Platform that consolidates engineering activity data and transforms it into useful development and delivery insights. The following specific objectives support it.

1. Develop a Go-based backend foundation with clear configuration management and structured logging.
2. Establish a reusable, well-tested HTTP integration client for communicating with the GitHub REST API, covering authentication, timeouts, error handling and pagination.
3. Collect repository, commit, pull-request and contributor information from GitHub.
4. Persist collected records in PostgreSQL in a way that avoids duplicates and supports incremental synchronisation.
5. Calculate clearly defined repository and development-activity metrics, and document the definition and limitations of each.

6. Expose analytics through documented REST endpoints.
7. Prepare a web dashboard that presents results to different engineering roles.
8. Use Docker and Docker Compose to provide a repeatable development and deployment environment.
9. Follow secure engineering practice for credentials and configuration.
10. Design the platform so that CI/CD analytics, DORA metrics and additional integrations can be added without redesigning the core.

## 1.9 Contributions

The contributions of this project can be grouped into conceptual, architectural, practical and educational categories.

- **Conceptual:** a clear problem formulation for fragmented engineering data and a four-version roadmap that moves from GitHub analytics through delivery analytics and DORA metrics to team intelligence.
- **Architectural:** a modular-monolith architecture with explicit module boundaries (integration, collector, analytics, scheduler, database, API) and an event-oriented long-term direction, together with documented reasons for the decisions deliberately postponed, such as microservices, message brokers and Kubernetes.
- **Design:** a proposed relational data model, precise metric definitions with their limitations, and a REST API design suitable for dashboards and reports.
- **Practical:** a working Go application foundation with configuration and logging, a container environment using Docker Compose and PostgreSQL, and a public source repository.
- **Operational and security:** a documented secret-leak incident and its remediation, which produced concrete rules for credential handling.
- **Educational:** a structured record of the Go runtime concepts studied during development, including compilation, goroutines, the G-M-P scheduler, and the difference between concurrency and parallelism.

An honest statement of implementation status is itself a contribution of this report. Throughout, the distinction between what has been built, what has been designed and what is only envisioned is stated explicitly, so that the report can be used as a faithful record of the project at the time of submission.

## 1.10 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature: software delivery research, developer-productivity measurement, CI/CD, existing engineering-analytics tools, architectural styles and the technologies used. Chapter 3 presents the proposed framework in detail: requirements, architecture, technology stack, data model, metric definitions, API and security design. Chapter 4 describes the experimentation carried out to date, the results obtained, the evaluation plan for the remaining work and a discussion of lessons learned. Chapter 5 concludes the report and outlines future work. References and appendices follow.

# Chapter 2
# Literature Review

## 2.1 Introduction

This chapter reviews the body of knowledge on which NorthStar is built. The review is organised around the questions a designer of an engineering intelligence platform must answer. What should be measured, and what does research say about which measures matter? What are the dangers of measuring software teams? How does a change flow through continuous integration and delivery, and what data does each stage produce? What tools already exist, and where do they fall short? Which architectural style suits a first release of such a platform, and which technologies are appropriate? The chapter ends by identifying the gap that NorthStar addresses.

## 2.2 Research on Software Delivery Performance

The most influential body of work on measuring software delivery is the DevOps Research and Assessment programme. Over several years, DORA surveyed tens of thousands of professionals and used statistical analysis to identify which practices and measures predict both software delivery performance and wider organisational performance. The results were summarised for practitioners in the book Accelerate by Forsgren, Humble and Kim and have been updated in annual State of DevOps reports [1], [2].

The central finding is that a small number of outcome measures characterise software delivery performance. Two measure the speed of delivery, deployment frequency and lead time for changes, and two measure its stability, change failure rate and the time to restore service after a failure. A key and counter-intuitive result is that speed and stability are not in opposition: the teams that deliver most frequently also tend to be the teams with the lowest failure rates and fastest recovery. Later reports have continued to refine the framework, including renaming restoration time as failed-deployment recovery time and discussing additional measures such as rework; for the purposes of this project the four classic measures remain the core of the DORA metrics version, and the definitions will be re-verified against the current DORA documentation when that version is implemented [2].

The DORA work also emphasises that these metrics describe a team or a service, not individuals, and that their value lies in observing trends and motivating improvement rather than in comparing teams in league tables. This point informs NorthStar’s design: metrics are presented with their definitions and limitations, and the platform avoids individual ranking.

**Table 2.1: DORA four key metrics and the data required to compute them**

| Metric | Question answered | Typical source data |
|---|---|---|
| Deployment frequency | How often do we release to production? | Deployment events with timestamps and status. |
| Lead time for changes | How long does a commit take to reach production? | Commit timestamp; deployment completion timestamp linked by commit SHA. |
| Change failure rate | What fraction of deployments cause a failure? | Deployment outcomes; incident or rollback records. |
| Failed-deployment recovery time | How long do we take to recover after a failed deployment? | Failure start and restoration timestamps. |

## 2.3 Measuring Developer Productivity

Measuring software teams is notoriously hazardous. Simple activity measures such as lines of code, commit counts or tickets closed are easy to collect, but they conflate activity with value and invite gaming. The SPACE framework proposed by Forsgren and colleagues argues that developer productivity is multidimensional and cannot be captured by any single metric. It identifies five dimensions, satisfaction and well-being, performance, activity, communication and collaboration, and efficiency and flow, and recommends that organisations use metrics from several dimensions together, combined with qualitative information [3].

The risk of over-reliance on metrics is a general phenomenon. Goodhart’s law, in the formulation popularised by Strathern, states that when a measure becomes a target, it ceases to be a good measure [19]. Muller develops the argument at length in The Tyranny of Metrics, describing how measurement fixation distorts behaviour and consumes effort [20]. Applied to software, a team that is rewarded for commit counts will split commits; a team that is rewarded for fast pull-request merging may reduce the quality of reviews.

These warnings translate into concrete design requirements for NorthStar. Every metric must have a precise definition. For example, “contributor activity” may mean commits authored, pull requests opened or pull requests merged, which are different measurements. The dashboard must show the definition and the limitations next to the number. The platform should emphasise team-level trends over individual comparison. The goal feature, in which managers define targets such as a deployment frequency above a threshold, a build success rate above ninety-five per cent or a lead time below twenty-four hours, must be presented as an aid for improvement conversations and not as a performance-management weapon.

## 2.4 Continuous Integration and Continuous Delivery

Continuous integration is the practice of merging developers' work into a shared mainline frequently, with each merge verified by an automated build and test run. Continuous delivery extends the practice so that every change that passes the pipeline is releasable, and continuous deployment goes further by releasing every passing change automatically. Humble and Farley's Continuous Delivery is the standard reference and introduces the deployment pipeline as the single path by which a change reaches production [4].

A deployment pipeline produces rich data: the time each stage starts and finishes, which stage failed, how long builds take, how often builds are retried, which commits were included in which release, and which environments received which artefacts. CI/CD systems such as Jenkins and GitHub Actions expose this data through APIs and webhooks [29], [30]. The DevOps Handbook describes how making this flow visible, through the "three ways" of flow, feedback and continual learning, is a prerequisite for improvement [5]. NorthStar's second planned version targets exactly this data: build history, build duration, success and failure, deployment history and pipeline trends.

Site reliability engineering contributes the operational half of the picture. The SRE literature introduces service-level indicators, objectives and error budgets and stresses blameless post-mortems and measurement of time to recovery [6]. These ideas underpin the change failure rate and recovery-time metrics of the DORA framework and explain why NorthStar's long-term roadmap includes integration with monitoring systems such as Prometheus [22].

## 2.5 Existing Engineering Analytics Tools

A number of commercial and open-source products already offer engineering analytics. Commercial offerings include engineering-management platforms such as LinearB, Jellyfish, Swarmia, Pluralsight Flow and Typo, many of which provide DORA dashboards, pull-request analytics, and sprint and investment reporting. Open-source efforts such as Apache DevLake and Google's Four Keys project aim to collect data from multiple tools into a unified data model and to compute DORA metrics. The reference screenshot used during the planning of NorthStar, a deployment-frequency, deployment-failure and time-to-build view from the Typo platform, illustrates the kind of insight these tools provide.

The descriptions in this section are intentionally general. The products evolve rapidly, and this report does not attempt to benchmark specific commercial features. Instead, the review highlights recurring characteristics, shown in Table 2.2, and the design space they leave open.

**Table 2.2: Characteristics of existing approaches and the NorthStar position**

| Approach | Strengths | Limitations relevant to this project |
|---|---|---|
| Native tool dashboards (GitHub Insights, CI dashboards) | No setup; authoritative for their own data. | Each shows only its own slice; no cross-tool joins; limited history and customisation. |
| Manual exports and spreadsheets | Flexible; cheap to start. | Slow, error-prone, not repeatable; definitions drift between analysts. |
| Commercial engineering-intelligence platforms | Polished, broad integrations, benchmarks. | Proprietary and priced per user; metric definitions are fixed by the vendor; not suitable as a learning vehicle. |
| Open-source data platforms | Open, extensible, community integrations. | Substantial setup complexity; broad scope can obscure the core design. |
| NorthStar (proposed) | Small, modular, transparent metric definitions; Go and PostgreSQL stack; designed for incremental growth. | Initially a single integration (GitHub); breadth of integrations and maturity are future work. |

Two observations follow. First, the existence of mature products validates the need for engineering analytics. Second, there is room for a transparent, modular implementation whose design can be studied end to end, from the HTTP client to the dashboard. NorthStar positions itself as such an implementation and, as a student project, also as a demonstration of the engineering skills required to build this class of system.

## 2.6 Architectural Styles

### 2.6.1 Monolith, Modular Monolith and Microservices

A monolithic application is deployed as a single unit. A microservice architecture decomposes the application into independently deployable services, each owning its data and communicating over the network. Newman's Building Microservices presents the benefits, independent scaling, independent deployment and technology heterogeneity, and also the costs, distributed-systems complexity, operational overhead and the difficulty of getting service boundaries right [16].

Fowler's "MonolithFirst" argument observes that almost all successful microservice systems started as monoliths that were later split, and that almost all systems built as microservices from the start ran into trouble because the boundaries were not yet understood [17]. A modular monolith is a single deployable unit whose internals are divided into modules with explicit boundaries; it keeps deployment and debugging simple while preserving the option to extract modules later.

The early planning notes for NorthStar described four major services: a collector service that collects data, a processing service that transforms it, an analytics service that computes metrics

and an API service that serves the dashboard. After analysis, these were adopted as module boundaries inside a single Go application for version 1 instead of separate deployables. The decision is justified by the project's scale: one developer, one integration, a single database and no independent scaling requirement. The modules can be promoted to services if scale or ownership requirements later demand it.

### 2.6.2 Event-Oriented Architecture

An event-oriented architecture structures a system around the production, detection and consumption of events. Hohpe and Woolf's Enterprise Integration Patterns catalogue the messaging patterns, channels, routers, translators and idempotent receivers, used to integrate heterogeneous systems [18]. Kleppmann discusses event logs, stream processing and the trade-offs of derived data in Designing Data-Intensive Applications [15].

For NorthStar the event model is attractive conceptually: pull_request.opened, pull_request.merged, build.completed and deployment.completed form a natural vocabulary, and metrics such as lead time are computed from the timestamps and relationships among them. However, introducing a message broker such as RabbitMQ, NATS or Kafka before there is an actual event-processing requirement would add infrastructure without adding value. The architectural decision recorded for the project is therefore to treat the event model as a long-term direction, to design internal data structures so that they can evolve toward it, and to postpone the broker.

### 2.6.3 REST

Representational State Transfer, introduced by Fielding, describes an architectural style for networked systems based on resources, uniform interfaces and stateless interactions [10]. REST over HTTP and JSON is the dominant style for web APIs, and it is the style used both by the GitHub API that NorthStar consumes and by the API that NorthStar will expose to its dashboard [11].

## 2.7 The Go Programming Language for Infrastructure Software

Go is a statically typed, compiled language designed at Google with emphasis on simplicity, fast compilation, garbage collection and built-in concurrency [7], [8]. Much of the cloud-native ecosystem, including Docker, Kubernetes, Prometheus and Terraform, is written in Go, which means that its libraries, idioms and tooling are well suited to infrastructure-oriented software.

Several properties make Go particularly appropriate for NorthStar. Compilation produces a single native executable that is easy to package in a small container image. The standard library includes a production-quality HTTP client and server, as well as JSON encoding, in the net/http and encoding/json packages [9]. Goroutines and channels make it straightforward to perform

many network requests concurrently, which matters when collecting data from many repositories.

Pike's well-known distinction between concurrency and parallelism is relevant here [26]. Concurrency is the composition of independently executing tasks; parallelism is the simultaneous execution of tasks. A program can be concurrent without being parallel. Go's runtime scheduler multiplexes many goroutines onto a smaller number of operating-system threads using the G-M-P model, in which G is a goroutine, M is a machine (an OS thread) and P is a processor, a logical resource required to execute Go code. This model is discussed further in Chapter 3 and Chapter 4, because understanding it was a significant learning outcome of the project.

It is also important to avoid overstating the benefits. Go does not automatically make a product better. The performance of a collector that talks to the GitHub API is dominated by network latency and by API rate limits, not by the language. The relevant advantages are the quality of the standard library, the ease of deploying a static binary and the ecosystem. The project therefore adopts the guideline of introducing concurrency where it provides a measurable benefit and not simply adding goroutines everywhere.

## 2.8 Persistence and Data Ingestion

PostgreSQL is an open-source relational database with a long record of reliability, rich SQL support, transactions with ACID guarantees, indexing, constraints and extensibility [12]. It is a good fit for structured engineering data such as repositories, commits and pull requests, and for time-based aggregation queries. Unique constraints and upsert statements (INSERT ... ON CONFLICT) allow repeated ingestion of the same source objects without creating duplicates.

Idempotency is a central concept for a collector that polls external systems. If the collector runs every few minutes and fetches overlapping data, the persistence layer must guarantee that processing the same source object twice leaves the database in the same state as processing it once. Kleppmann discusses idempotence as one of the foundations of reliable data systems [15]. In NorthStar this is achieved by storing the source system's identifier, such as the commit SHA or the pull-request number within a repository, and enforcing uniqueness on it.

The GitHub REST API imposes practical constraints that shape the collector: authentication through tokens, pagination of list endpoints (with a maximum page size of one hundred items), primary and secondary rate limits that are communicated through response headers, and support for conditional requests using entity tags [11]. The collector must therefore be incremental: it should request only data changed since the last successful synchronisation where the API allows it, and it must respect and react to rate-limit information.

## 2.9 Containerisation and Configuration

Containers package an application with its dependencies and run it in an isolated user-space environment that shares the host's kernel. Merkel's article on Docker explains how containers make deployments consistent across development and production environments [14], and the Docker documentation describes images, containers, volumes, networks and Compose [13]. For NorthStar, Docker Compose is used to run the Go application and a PostgreSQL 17 container together, with a named volume to persist database storage.

The Twelve-Factor App methodology recommends storing configuration in environment variables, strictly separating configuration from code, and treating backing services such as databases as attached resources addressed through URLs [34]. NorthStar follows these recommendations: the GitHub token, the database URL, the port and the polling interval are supplied through the environment, and a .env.example file with placeholders, not a real .env file, is committed to the repository.

Kubernetes, the dominant container orchestrator, provides scheduling, scaling and self-healing for containerised workloads [24]. It is a natural long-term deployment platform for NorthStar but is explicitly not required for the first release. Introducing Kubernetes before the application works reliably under Docker would add complexity without adding capability.

## 2.10 Observability

Observability is the ability to understand the internal state of a system from its external outputs. Prometheus is an open-source monitoring system that scrapes time-series metrics and supports alerting; Grafana is a visualisation platform commonly used with Prometheus [22], [23]. In the long-term vision of NorthStar these tools are both data sources, since operational metrics and incident data feed the change-failure and recovery-time metrics, and potential deployment targets for NorthStar's own operational metrics. For version 1, observability is limited to structured logging.

## 2.11 Research Gap and Positioning

The literature establishes what should be measured (DORA metrics and multidimensional productivity measures), what dangers to avoid (metric fixation and individual ranking), how delivery pipelines generate data (CI/CD and SRE practice) and which architectural and technology choices are sensible for a first implementation (modular monolith, Go, PostgreSQL, Docker). Existing products show that the problem is real and that solutions are viable.

What is less often available, particularly to students and small teams, is a transparent reference implementation whose architecture, metric definitions and trade-offs are fully documented and incrementally built. NorthStar addresses this gap by combining a clear product vision with an

engineering approach that deliberately avoids premature complexity and records the reasoning behind each decision.

## 2.12 Summary

This chapter reviewed research on software-delivery performance, the pitfalls of measuring developer productivity, CI/CD and SRE practice, existing tools, architectural styles and the technologies used in the project. The principal conclusions that shape the next chapter are that metrics must be precisely defined and carefully presented, that collection should be separated from calculation, that a modular monolith is the appropriate starting architecture, that idempotent incremental ingestion is essential, and that the first release should concentrate on a single, complete, end-to-end workflow.

# Chapter 3
# Proposed Model/Framework

## 3.1 Design Philosophy

The proposed framework is guided by a small set of principles that were agreed during the planning of the project and that explain many of the design decisions in this chapter.

1. **Start small and expand incrementally.** The first release must be a complete, working product that does one thing well, not a large architecture that only starts successfully.
2. **Consume, never create.** NorthStar reads data from the tools that teams already use. Those tools remain the systems of record.
3. **Separate collection from calculation.** The collector gathers raw data and does not calculate. The analytics engine calculates. The API serves precomputed results.
4. **Organise by business capability.** Packages are named for what they do, such as integrations, collector and analytics, and generic layers such as controllers, services and models are not created merely because other projects use them. Abstractions are introduced only when they solve a real problem.
5. **Precise, honest metrics.** Every metric has a definition and documented limitations, and the platform does not rank individuals.
6. **Distinguish implemented from planned.** Documentation, the README and this report state clearly what is built and what is only designed.

## 3.2 The Six Pillars of NorthStar

The conceptual design of the platform rests on six pillars. They describe what the platform does, independently of the technologies used to implement it.

**Table 3.1: The six pillars of NorthStar**

| # | Pillar | Description |
|---|---|---|
| 1 | Source integration | Answers “where does the data come from?” Authenticates with external systems such as GitHub, GitLab, Jenkins and Jira and fetches data. NorthStar never creates data; it consumes it. |
| 2 | Analytics engine | “The brain.” Calculates DORA metrics, lead time, deployment frequency, build success rate, review time and other measures from stored raw data. |
| 3 | Intelligence layer | Rule-based (not AI) analysis that detects bottlenecks, detects failing pipelines, identifies deployment slowdowns, compares teams and produces weekly insights. |

| # | Pillar | Description |
|---|---|---|
| 4 | Dashboards | Different dashboards for different personas: executive, team, developer, deployment and DORA dashboards. |
| 5 | Reports | Generated outputs such as weekly engineering reports, monthly delivery reports, sprint summaries, quarterly DORA metrics and daily workflow summaries. |
| 6 | Goals | Managers define targets, for example deployment frequency above twenty per week, build success rate above ninety-five per cent, and lead time below twenty-four hours, and the platform tracks progress against them. |

The distinction between the analytics engine and the intelligence layer deserves emphasis. The analytics engine produces numbers. The intelligence layer interprets them using explicit, explainable rules, for example flagging a pipeline whose failure rate over the last seven days exceeds a configured threshold, or a repository whose median pull-request review time has doubled compared with the previous period. The planning notes specifically marked this layer as “not AI”. AI-assisted summaries are a possible later extension, but only when supported by reliable, traceable underlying data.

## 3.3 Users, Dashboards and Reports

Each persona identified in Chapter 1 is served by a dedicated dashboard so that a number or chart that matters to one role does not clutter the view of another. The project structure reserves a Dashboard module with separate views for the CTO, the team lead, the executive audience and other roles.

**Table 3.2: Dashboards and their principal content**

| Dashboard | Primary audience | Representative content |
|---|---|---|
| Executive dashboard | CTO, executives | Organisation-wide delivery health, trends, goal attainment, engineering health score (later versions). |
| Team dashboard | Engineering managers, team leads | Team activity, pull-request flow, review times, bottlenecks, sprint progress. |
| Developer dashboard | Developers | Own pull requests, review queues, contribution patterns (for self-reflection, not ranking). |
| Deployment dashboard | DevOps engineers | Deployment frequency, deployment failures, build duration, workflow status. |
| DORA dashboard | Managers, DevOps, SRE | Four key metrics with trends and targets. |

Reports complement dashboards for audiences that prefer periodic summaries. The planned report types are daily workflow reports, weekly engineering reports, sprint summaries, monthly delivery reports and quarterly DORA reports. Reports are generated from the same precomputed metrics as the dashboards, which guarantees that the numbers in a report match the numbers in the corresponding dashboard.

## 3.4 Requirements

### 3.4.1 Functional Requirements

**Table 3.3: Functional requirements for the initial release**

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | Load application configuration from environment variables. | High |
| FR-02 | Initialise application logging. | High |
| FR-03 | Authenticate with the GitHub API securely using a token. | High |
| FR-04 | Retrieve repository information. | High |
| FR-05 | Retrieve commit and pull-request data, handling pagination. | High |
| FR-06 | Collect contributor information. | Medium |
| FR-07 | Persist collected records in PostgreSQL without duplicates. | High |
| FR-08 | Calculate basic repository activity metrics. | High |
| FR-09 | Expose analytics through REST endpoints. | High |
| FR-10 | Refresh data periodically through a scheduler. | Medium |
| FR-11 | Present analytics in a web dashboard. | Medium |

### 3.4.2 Non-Functional Requirements

**Table 3.4: Non-functional requirements**

| Category | Requirement |
|---|---|
| Security | Secrets are supplied through environment configuration and excluded from source control; tokens are never logged; tokens use minimum scopes. |
| Reliability | HTTP requests use timeouts; non-success responses are handled; transient failures are retried with bounded backoff. |
| Maintainability | Modules have clear responsibilities and limited coupling; main.go contains no business logic. |
| Extensibility | New integrations fit into the integrations package without changes to analytics or the API. |
| Performance | Collection avoids unnecessary repeated requests, supports pagination and incremental synchronisation. |
| Observability | Logs help diagnose startup and integration failures; they are informative but never contain secrets. |

| Category | Requirement |
|---|---|
| Portability | Docker and Docker Compose provide a repeatable environment on Linux, Windows (WSL) and macOS. |
| Transparency | Each metric is documented with definition, formula and limitations. |

## 3.5 System Architecture

### 3.5.1 Long-term Architecture

The long-term architecture is event-oriented and shown in Figure 3.1. External systems provide data. Integrations authenticate and fetch it; an event-collection layer receives it through polling and webhooks; a processing layer validates and normalises it; the analytics engine computes metrics; the storage layer persists records and metric snapshots; and a REST API serves dashboards, reports and insights. This is the conceptual pipeline for the full product; it was not implemented in its entirety in the first sprint and is not a claim of completed functionality.

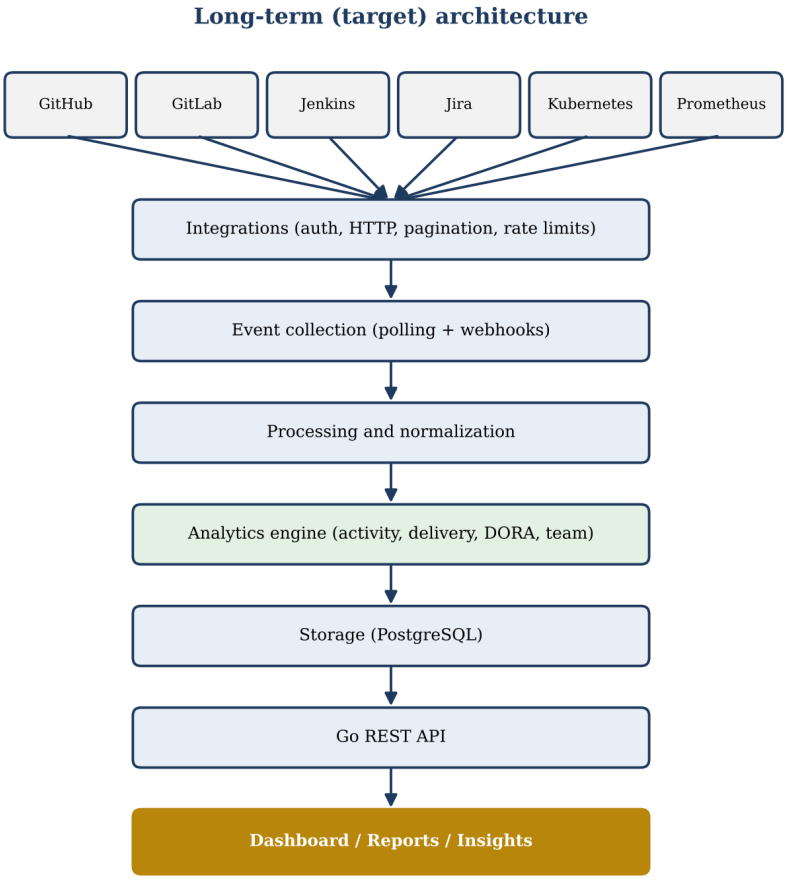



**Figure 3.1: Long-term (target) architecture of NorthStar**

### 3.5.2 Version 1 Architecture

For version 1 the architecture is intentionally simpler: a modular Go application that flows from the GitHub API through the integration module and the collector into analytics, PostgreSQL, the Go REST API and finally the dashboard (Figure 3.2). The difference between the long-term architecture and the version 1 architecture matters, because it expresses the decision to build a working product first and expand the platform afterward.

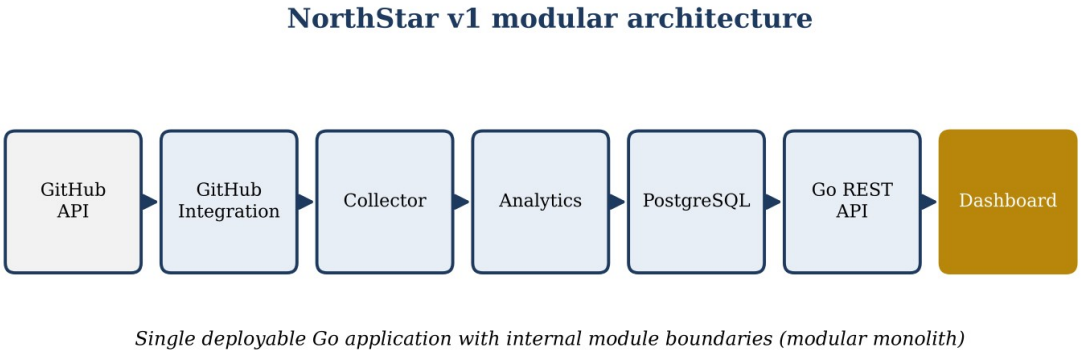



**Figure 3.2: NorthStar version 1 modular architecture**

**Table 3.5: Responsibilities of the version 1 components**

| Component | Responsibility |
|---|---|
| Integration (GitHub) | Communicate with the GitHub REST API: authentication, requests, pagination, response decoding and error handling. |
| Collector | Retrieve and coordinate data collection across repositories and endpoints; hand validated records to persistence. Contains no HTTP details and no metric calculations. |
| Analytics | Calculate basic metrics from stored records and write metric snapshots. |
| Scheduler | Trigger collection and analytics at configured intervals. |
| Database | Persist collected data and metric snapshots; provide query operations. |
| API | Make stored data and metrics accessible to the frontend over REST/JSON. |
| Dashboard | Present metrics and insights to users. |

### 3.5.3 Mapping the Original Four Services to Modules

The early design notes proposed four major services. They map directly onto the modules of the modular monolith, which keeps the original conceptual separation while avoiding premature distribution.

**Table 3.6: Original services and corresponding version 1 modules**

| Original service | Purpose | Version 1 module |
|---|---|---|
| Collector service | Collect data | internal/integrations/github and internal/collector |
| Processing service | Transform data | Normalisation mapping inside the collector and database packages |
| Analytics service | Compute metrics | internal/analytics |
| API service | Serve the dashboard | internal/api |

The notes also record a high-level flow: the user interacts with a dashboard; the dashboard calls the NorthStar API; the API reads from the analytics layer and database, and a calculator or collector engine reads from GitHub, Jenkins and Jira. This is consistent with the target architecture and is simplified, in version 1, to a single integration.

## 3.6 Repository and Project Structure

A considerable amount of design effort went into the repository structure, with the objective of making it look and behave like a real software project without introducing unnecessary abstraction. The agreed structure is shown below. It is the intended structure; directories are added when the corresponding implementation begins, and an empty directory does not imply a completed feature.

```
NorthStar/
├── main.go                  # composition root: wires and starts the app
├── go.mod / go.sum
├── README.md
├── .env.example             # placeholders only - never real secrets
├── .gitignore / .dockerignore
├── Dockerfile               # multi-stage Go build
├── docker-compose.yml       # app + PostgreSQL
├── documentation/           # runbooks, architecture, roadmap
├── assets/                  # logo, screenshots
├── config/                  # configuration loading
├── internal/
│   ├── api/                 # REST handlers
│   ├── integrations/
│   │   └── github/          # client, models, repositories,
│   │                        # commits, pullrequests, contributors
│   ├── collector/           # coordinates data collection
│   ├── analytics/           # metric calculation
│   ├── scheduler/           # periodic jobs
│   ├── database/            # persistence
│   └── logger/              # logging
├── web/                     # dashboard frontend
├── deployments/             # deployment manifests
├── tests/
└── .github/workflows/       # CI / CD pipelines
```

*Listing 3.1: Intended repository structure of NorthStar*

### 3.6.1 Rationale for Key Structural Decisions

- **Root-level main.go instead of cmd/server/main.go.** Version 1 has one executable and its entry point should be obvious. If NorthStar later has multiple executables, a cmd/ directory can be introduced.
- **The internal/ directory.** Go restricts imports of internal packages to code within the parent tree, which keeps implementation details private to the application.
- **integrations/github/ instead of internal/github/.** The future product will integrate several systems (GitLab, Jira, Jenkins, Kubernetes). Grouping them under integrations/ organises code by the external system with which it communicates.
- **No generic controllers, services and repositories folders.** Organising by capability keeps the first version understandable, and abstractions are added only when a real problem demands them.

The planning notes also sketched a structure that included separate directories for dashboards per persona, reports by frequency (daily, weekly, monthly, quarterly, sprint), authentication modules for Git and Jira, an APIs directory, a workflow directory for CI and CD, and a tests directory. These ideas are retained as the longer-term organisation of the web and reporting modules, while the version 1 structure shown above concentrates on the backend.

## 3.7 Technology Stack

The technology stack combines selected technologies, which are in use or being set up, with planned and future technologies. Implementation status is stated explicitly to avoid overclaiming.

**Table 3.7: Technology stack, purpose and status**

| Layer | Technology | Purpose | Status |
|---|---|---|---|
| Primary backend | Go | Core application, integrations and REST API | Selected; project initialised |
| Supporting backend | Python | Possible analytics and ML/AI processing | Future |
| Database | PostgreSQL 17 | Persistent storage of engineering data | Container setup completed |
| DB access | SQL / Go database library | Database interaction | Not finalised |
| Frontend | React + TypeScript | Analytics dashboard | Planned |
| Containers | Docker | Package application and dependencies | Initial setup completed |

| Layer | Technology | Purpose | Status |
|---|---|---|---|
| Local orchestration | Docker Compose | Run application and PostgreSQL together | Initial setup completed |
| Cache | Redis | Caching or background-work support | Future |
| Messaging | RabbitMQ or NATS | Asynchronous event processing | Future |
| Orchestration | Kubernetes | Deployment and scaling | Future |
| CI/CD | GitHub Actions | Build, test and publish automation | Planned for NorthStar |
| Monitoring | Prometheus + Grafana | Operational observability | Future integration |

The planning notes list ten technologies in order of intended use: Go (backend), Python (analytics), React (frontend), PostgreSQL (database), Redis (cache), RabbitMQ (message queue), Prometheus (metrics), Grafana (visualisation), Docker (container) and Kubernetes (orchestration). The refined plan in Table 3.7 keeps all ten as parts of the long-term vision but is explicit about which are in use today. The exact Go database library, the frontend implementation, the message broker, the cache architecture and the deployment platform had not been finalised at the time of this report.

### 3.7.1 Why Go?

Go was chosen as the primary language because NorthStar is, at its heart, a network-intensive infrastructure application. Go is fast, compiles to a native executable, has a strong standard library for HTTP and JSON, produces small deployment artefacts and has built-in support for concurrent I/O through goroutines [7], [8], [9]. The surrounding ecosystem of Docker, Kubernetes, Prometheus and Terraform means that engineers who learn Go also learn the idioms of cloud-native software. A conscious decision was made that NorthStar should be a substantial Go project and not a small Go API attached to a primarily Python application; Python is reserved for possible future analytics or machine-learning extensions.

### 3.7.2 Why PostgreSQL?

PostgreSQL provides relational storage, transactions, constraints, indexing and a mature ecosystem [12]. Engineering data is naturally relational: a repository has many commits and pull requests, and a contributor authors many commits. Time-range aggregations, joins and window functions are natural in SQL. PostgreSQL is also the only data store required for version 1, in line with the principle that additional infrastructure such as Redis or a message broker should be introduced only when a real need appears.

### 3.7.3 Why Docker?

Docker provides a repeatable environment for the application and its dependencies [13], [14]. A multi-stage Dockerfile compiles the Go application in a build image and copies the resulting binary into a small runtime image. Docker Compose runs the application together with a PostgreSQL 17 container, a named volume persists database storage and environment variables supply configuration.

## 3.8 GitHub Integration Design

The GitHub integration is the first and most important module, because every other component depends on reliable access to GitHub data. It lives in internal/integrations/github and is divided by resource: client.go, models.go, repositories.go, commits.go, pullrequests.go and contributors.go. Development starts with client.go, since every other GitHub operation uses it.

### 3.8.1 The API Client

The client is a small struct that holds a reusable http.Client, the authentication token and the base URL. Reusing a single http.Client allows connection pooling, and setting an explicit timeout prevents a slow or unresponsive server from blocking the application indefinitely. The principal design decisions are summarised in Table 3.8.

**Table 3.8: Design decisions for the GitHub client**

| Detail | Decision |
|---|---|
| Base URL | https://api.github.com |
| User-Agent | NorthStar/1.0 (GitHub requires a User-Agent header) |
| Timeout | 30 seconds on the shared http.Client |
| Authentication | Bearer token supplied through configuration |
| Accept header | GitHub JSON media type (application/vnd.github+json) |
| HTTP client | Single reusable http.Client, enabling connection reuse |

The following listing shows the design of the client as agreed during development. It is a design listing; the repository should be consulted for the current state of implementation.

```
package github

type Client struct {
    httpClient *http.Client
    token      string
    baseURL    string
}

// NewClient only constructs - it never performs network I/O.
func NewClient(token string) *Client {
    return &Client{
        httpClient: &http.Client{Timeout: 30 * time.Second},
        token:      token,
        baseURL:    "https://api.github.com",
    }
}

// newRequest builds a request and attaches headers + auth.
func (c *Client) newRequest(method, endpoint string) (*http.Request, error)

// do executes a request, checks the status code and decodes JSON.
func (c *Client) do(req *http.Request, target interface{}) error

// Ping verifies connectivity and authentication via GET /user.
func (c *Client) Ping() error
```

*Listing 3.2: Design of the GitHub client (signatures)*

Two helper methods carry most of the logic. newRequest creates an HTTP request and attaches the standard headers and the authentication token, so that no other function has to remember them. do executes the request, closes the response body, checks the HTTP status code, and decodes the JSON body into a target structure. Centralising this behaviour gives one place in which to add retry logic, rate-limit handling and logging later. The Ping method calls GitHub's authenticated /user endpoint to verify that the token works.

An important design principle is that the constructor must not perform network calls. A constructor that unexpectedly contacts a remote server is hard to test and surprising to callers. Connectivity checking therefore belongs in main.go, which orchestrates startup:

```
client := github.NewClient(cfg.GitHubToken)
if err := client.Ping(); err != nil {
    log.Fatal(err)
}
log.Println("GitHub Connection Established")
```

*Listing 3.3: Startup connectivity check in main.go*

### 3.8.2 Models and Resource Operations

After the client, the next step is to define Go structures, models.go, into which GitHub's JSON responses are decoded. This step teaches how JSON field names map to Go struct fields and keeps GitHub-specific shapes separate from NorthStar's internal records. The resource files then implement individual operations using the shared client, as summarised in Table 3.9.

**Table 3.9: GitHub endpoints used by the version 1 collector**

| Operation | GitHub REST endpoint | Purpose |
|---|---|---|
| Verify access | GET /user | Check token validity (Ping). |
| Repository details | GET /repos/{owner}/{repo} | Repository metadata. |
| Commits | GET /repos/{owner}/{repo}/commits | Commit history, paginated. |
| Pull requests | GET /repos/{owner}/{repo}/pulls?state=all | PR lifecycle: created, merged, closed. |
| Contributors | GET /repos/{owner}/{repo}/contributors | Contributor information. |
| Workflow runs (v2) | GET /repos/{owner}/{repo}/actions/runs | Build history for CI/CD analytics. |
| Deployments (v2/v3) | GET /repos/{owner}/{repo}/deployments | Deployment events for DORA metrics. |

## 3.9 Data Collection Strategy

The collector is responsible for deciding what to fetch and when, and for handing results to the persistence layer. It must not contain HTTP details, which belong to the integration module, nor metric calculations, which belong to analytics.

### 3.9.1 Polling and Scheduling

Version 1 uses controlled polling. The scheduler triggers a collection run at a configurable interval; the planning notes suggested five minutes as a starting point. In each run the collector requests commits, pull requests and related data for each connected repository. Webhooks, in which GitHub pushes events to NorthStar as they occur, can complement or replace polling in later versions, since they provide lower latency and fewer wasted requests, but they require a publicly reachable endpoint and signature verification, which are out of scope for the first release.

### 3.9.2 Pagination, Rate Limits and Retries

List endpoints return at most one hundred items per page, so the collector must follow pagination links until all pages are consumed or until it reaches data that has already been collected. GitHub applies rate limits to authenticated requests and reports the remaining allowance and the reset time in response headers [11]. The client should read these headers, pause when the allowance is exhausted, and avoid hammering the API. Transient failures, such as network errors or server errors, are retried a bounded number of times with exponential backoff, while client errors such as an invalid token are reported immediately.

### 3.9.3 Incremental Synchronisation and Idempotency

Repeated collection must not create duplicates or waste requests. Two mechanisms address this. First, the collector records the time of the last successful synchronisation for each repository and resource in the collection_runs table, and uses it as a starting point where the API supports a since parameter or ordered results. Second, the persistence layer uses source identifiers, the commit SHA for commits and the pull-request number within a repository for pull requests, with unique constraints and upsert statements, so that processing the same object twice yields the same stored state [15].

### 3.9.4 Data Normalisation

Normalisation converts source-specific JSON into NorthStar's internal representation. For example, GitHub represents a commit's author both as a Git identity (name and email) and, if the email can be matched, as a GitHub account. NorthStar stores both, and does not assume they are the same. Timestamps are converted to a timezone-aware format and stored in UTC. Fields that may be absent, such as the merged-at time for an unmerged pull request, are stored as nullable values and not as sentinel values.

## 3.10 Data Model

The database schema is designed around the data that is actually collected, instead of inventing a large schema in advance. Figure 3.3 and Table 3.10 describe the proposed logical model. It is a design element; the final schema will follow the verified GitHub API models.

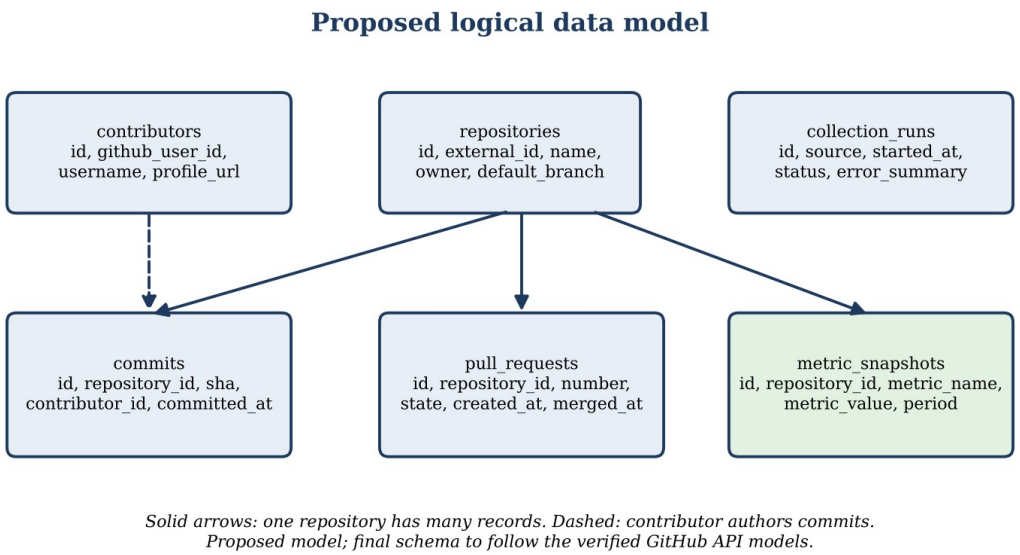



**Figure 3.3: Proposed logical data model**

**Table 3.10: Proposed data entities**

| Entity | Example attributes | Purpose |
|---|---|---|
| repositories | id, external_id, name, owner, default_branch, created_at | Identify connected repositories. |
| contributors | id, github_user_id, username, profile_url | Represent repository contributors. |
| commits | id, repository_id, sha, contributor_id, message, committed_at | Store commit activity. |
| pull_requests | id, repository_id, number, title, state, created_at, merged_at, closed_at | Store pull-request lifecycle data. |
| collection_runs | id, source, started_at, completed_at, status, error_summary | Track collection executions and failures. |
| metric_snapshots | id, repository_id, metric_name, metric_value, period_start, period_end | Store calculated metric values. |

### 3.10.1 Integrity Considerations

- Use source-system identifiers with unique constraints to prevent duplicates.
- Use foreign keys to maintain relationships between repositories and their activity records.
- Store all timestamps in a timezone-aware format, normalised to UTC.
- Record collection status and error information to support troubleshooting and incremental synchronisation.
- Wrap related writes in transactions so that a failed run does not leave partial data.
- Add indexes on columns used in time-range queries, such as committed_at and merged_at.

### 3.10.2 Database Connection URL

The application connects to PostgreSQL through a connection URL. The general format recorded in the planning notes is shown below, followed by a concrete example for local development.

```
postgres://USERNAME:PASSWORD@HOST:PORT/DATABASE?sslmode=disable

postgres://postgres:postgres@localhost:5432/northstar?sslmode=disable
  protocol  user      password  host      port db       ssl option
```

*Listing 3.4: PostgreSQL connection URL format and local example*

When the application itself runs inside a Docker Compose network, the host in the URL must be the Compose service name (postgres) and not localhost, because inside the NorthStar container localhost refers to the NorthStar container itself and not to the database container.

This networking lesson is discussed in Chapter 4. The sslmode=disable option is acceptable only for local development; production deployments should use encrypted connections.

## 3.11 Metric Definitions

Precise definitions are a design requirement. This section defines the metrics planned for each version, together with their limitations. The definitions will be implemented as documented functions in the analytics module and displayed next to the numbers in the dashboards.

### 3.11.1 Version 1: Engineering Activity Metrics

**Table 3.11: Version 1 activity metrics**

| Metric | Definition | Limitation |
|---|---|---|
| Commits in period | Number of distinct commits with committed_at in [start, end). | Counts activity, not value or productivity; commit size varies widely. |
| Pull requests opened | Number of pull requests with created_at in the period. | Sensitive to PR-splitting practices. |
| Pull requests merged | Number of pull requests with merged_at in the period. | Excludes closed-unmerged PRs. |
| Open vs closed PRs | Counts of PRs by state at the end of the period. | Snapshot; does not show how long PRs stayed open. |
| Average time to merge | Mean of (merged_at - created_at) for PRs merged in the period. | Mean is skewed by outliers; the median and percentiles should be shown too. |
| Commit activity over time | Commits grouped by day or week. | Time-zone and batching effects (for example squash merges). |
| Contributor activity | Per contributor: commits authored, PRs opened or PRs merged, each reported separately. | The three measures differ; no single score is computed; not for ranking individuals. |

### 3.11.2 Version 2: Delivery Metrics

- **Build success rate** = successful builds / total completed builds in the period. The planned goal in the notes is a rate above ninety-five per cent.
- **Build duration** = completed_at minus started_at for each workflow run, summarised by median and ninety-fifth percentile.
- **Pipeline trend** = build success rate and duration plotted over time.
- **Deployment history** = chronological list of deployments with environment, status and associated commit.

### 3.11.3 Version 3: DORA Metrics

**Table 3.12: Planned formulas for the DORA metrics**

| Metric | Formula | Example goal |
|---|---|---|
| Deployment frequency | Number of successful production deployments divided by the number of days or weeks in the period. | More than 20 per week (from the project notes). |
| Lead time for changes | Median of (deployment completed_at - commit committed_at) over changes in the period. | Less than 24 hours. |
| Change failure rate | Failed or remediated deployments divided by total deployments in the period. | Team-defined target. |
| Failed-deployment recovery time | Mean or median of (restored_at - failure_detected_at) for failed deployments. | Team-defined target. |

Two practical difficulties are recognised. Linking a commit to the deployment that shipped it requires a reliable association, usually through commit SHAs recorded on deployments. Identifying failures and recovery requires an incident or rollback signal, which comes from deployment status, monitoring or an issue tracker. These dependencies are the reason DORA metrics follow, and do not precede, the GitHub and CI/CD integrations.

### 3.11.4 Version 4: Team Intelligence

Version 4 adds team comparison, an engineering health score, sprint analytics, an executive dashboard and historical trends. The engineering health score, a composite of several normalised metrics, is the most delicate item because composites hide detail and can be misinterpreted. The design intention is to present the components of the score alongside the score itself and to document exactly how it is computed.

## 3.12 Analytics Engine and Intelligence Layer

The analytics engine follows a simple pipeline recorded in the design notes: raw commits, raw builds and raw deployments are read from storage; the engine calculates lead time, deployment frequency and build success; and the results are stored as analytics. The dashboard later asks the API for already calculated metrics. Because calculation is separated from collection, metrics can be recomputed whenever a definition changes without refetching data from the external systems.

The intelligence layer sits on top of the analytics engine and applies explicit rules. Examples of the rules planned are listed below. Because they are rules and not opaque models, each insight can explain why it was raised.

- **Bottleneck detection:** flag a repository when the median pull-request review time exceeds a configured threshold or has increased by more than a configured percentage relative to the previous period.
- **Failing pipeline detection:** flag a workflow whose failure rate over the last N runs exceeds a threshold.
- **Deployment slowdown:** flag when deployment frequency drops below a fraction of its trailing average.
- **Team comparison:** present normalised metrics for teams side by side, with context, and without ranking.
- **Weekly insights:** summarise the most significant changes in the metrics during the week.

## 3.13 REST API Design

The Go REST API, built on the standard net/http package, exposes stored data and calculated metrics to the dashboard. The principle recorded in the notes is that the dashboard calls an endpoint such as GET /dashboard and receives already calculated metrics. A planned set of endpoints for version 1 is shown in Table 3.13.

**Table 3.13: Planned REST endpoints for version 1**

| Method | Path | Description |
|---|---|---|
| GET | /health | Liveness check of the application and database connection. |
| GET | /api/v1/repositories | List connected repositories. |
| GET | /api/v1/repositories/{id} | Repository details and summary metrics. |
| GET | /api/v1/repositories/{id}/commits | Commits in a period, with pagination. |
| GET | /api/v1/repositories/{id}/pull-requests | Pull requests in a period, filtered by state. |
| GET | /api/v1/repositories/{id}/contributors | Contributor activity, with metric definitions. |
| GET | /api/v1/repositories/{id}/metrics | Calculated activity metrics for a selected period. |
| GET | /api/v1/dashboard | Precomputed metrics used by the dashboard. |

Responses are JSON. Each metric response includes the metric name, the value, the period, and a short definition so that the dashboard can display the definition beside the value. Errors use appropriate HTTP status codes and a consistent error body. Input such as date ranges is validated, and pagination parameters are bounded.

## 3.14 Application Startup and Composition Root

The main.go file is a composition root. Its job is to initialise and connect components, not to contain the application's business logic. The principle agreed is that main.go should explain how NorthStar starts, not how NorthStar works. The planned startup sequence is shown in Figure 3.4.

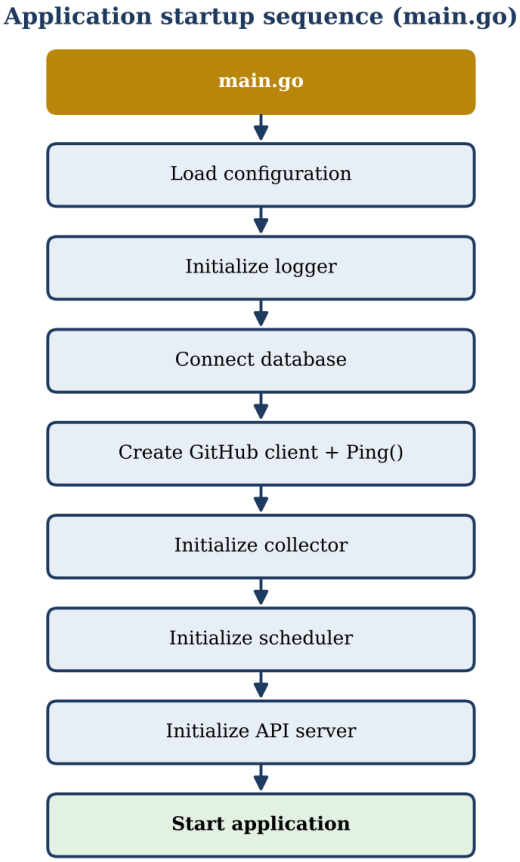



**Figure 3.4: Planned application startup sequence**

This arrangement makes the application testable, because each component can be constructed independently with test doubles, and readable, because a new developer can understand the entire startup order in a few lines.

## 3.15 End-to-End Workflow

The complete workflow of version 1 is illustrated in Figure 3.5 and corresponds to the workflow recorded in the design notes. The user connects GitHub by providing a token through the dashboard or configuration. The collector starts and, at each interval, requests commits, pull requests and workflow information and stores raw data. The analytics engine reads raw data,

calculates metrics and stores them. When the dashboard asks the API for data, the API returns already calculated metrics.

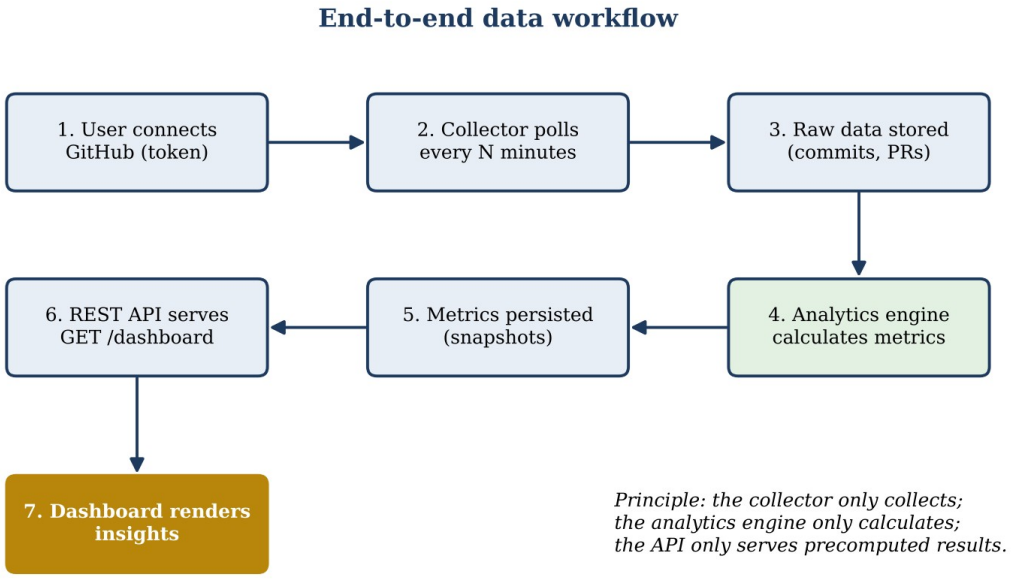



**Figure 3.5: End-to-end data workflow**

The end-to-end demonstration of version 1 is defined as one complete workflow: connect GitHub, select a repository, collect commits, pull requests and contributor information, store them in PostgreSQL, calculate basic metrics, expose them through the API and display them in a dashboard. This definition of done keeps the scope focused.

## 3.16 Configuration and Secrets Management

Following the twelve-factor approach, configuration is loaded from environment variables, validated at startup and held in a typed structure [34]. The configuration structure designed for the application has the following shape:

```
type Config struct {
    Port         string
    GitHubToken  string
    DatabaseURL  string
    PollInterval time.Duration
}
```

*Listing 3.5: Configuration structure*

The loader reads each variable, applies defaults where sensible (for example, a default port), fails with a clear error if a required variable such as the GitHub token is missing, and parses

values such as the polling interval into the proper type. The .env file is excluded from version control, and .env.example is committed with placeholder values only.

```
PORT=8080
GITHUB_TOKEN=your_token_here
DATABASE_URL=postgres://user:password@postgres:5432/northstar?sslmode=disable
POLL_INTERVAL=5m
```

*Listing 3.6: Example .env.example with placeholders*

## 3.17 Containerisation Design

The Dockerfile uses a multi-stage build. The first stage uses a Go toolchain image to download modules and compile the application into a static binary. The second stage copies the binary into a minimal runtime image. The result is a small image that contains no compiler, source code or build cache. Docker Compose defines two services, the NorthStar application and PostgreSQL 17, a named volume for PostgreSQL data and the environment variables for configuration. The two services share a Compose network, on which the database is reachable by its service name.

Two practical Docker lessons arose during development. First, the difference between docker compose down and docker compose down -v: the first removes containers and networks but preserves named volumes, while the second also removes the volumes and can therefore delete stored PostgreSQL data. Second, the host-name rule described above, in which the database host in a Compose environment is the service name, not localhost.

## 3.18 Security Design

### 3.18.1 Credential Management

GitHub credentials are supplied through environment variables or a suitable secrets mechanism. Real .env files are never committed. Tokens are given the minimum scopes required, for example read access to repository metadata, contents and pull requests for version 1. The application never logs the token and redacts it from error messages. GitHub's push protection and secret scanning provide an additional safety net by blocking pushes that contain recognised secrets [28].

### 3.18.2 API and Application Security

- Validate and bound all input to the REST API; use parameterised queries for all database access to prevent SQL injection.
- Use timeouts on HTTP servers and clients to limit the effect of slow or malicious peers.
- Run the container as a non-root user in production-oriented images.
- Use TLS for any non-local deployment, including database connections.

- Plan authentication and role-based access for the dashboard as a later extension; version 1 deliberately excludes a sophisticated authentication system.

## 3.19 Concurrency Design

NorthStar will communicate with external APIs, collect data and process records, and eventually handle several integrations and repositories. This makes concurrency useful: collecting data from several repositories concurrently is faster than waiting for each request to finish sequentially. Go's runtime makes this easy, but the design guidance is to introduce concurrency only when it provides a real benefit.

The understanding gained during the project of how Go executes concurrent code is summarised in the G-M-P model, depicted in Figure 3.6. A goroutine (G) is a lightweight unit of execution managed by the Go runtime. An operating-system thread is called a machine (M). A processor (P) is a logical scheduling resource that a thread must hold in order to execute Go code; it is not a physical CPU core. Goroutines wait in run queues attached to processors. When a goroutine blocks, for example on network I/O, the runtime parks it and lets the thread run another runnable goroutine. Idle processors can steal work from other processors' queues to balance load.

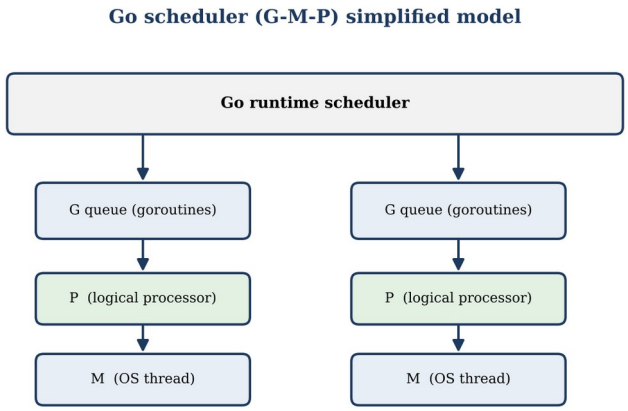



**Figure 3.6: Simplified model of the Go scheduler (G-M-P)**

For NorthStar's collector the practical consequence is that a bounded pool of worker goroutines, each handling a repository, can fetch data concurrently while the runtime multiplexes the goroutines onto a small number of threads. The pool must be bounded so that the program does not exceed GitHub's rate limits, and a context should be passed to every request so that work can be cancelled when the application shuts down.

## 3.20 Testing and Documentation Framework

Testing is designed alongside implementation and is described in detail in Chapter 4. Documentation is kept in the repository alongside the code and not only in conversations. The planned documents are README.md (what NorthStar is and how to run it), GO_RUNBOOK.md (Go concepts relevant to the project), SYSTEM_ARCHITECTURE.md, PROJECT_STRUCTURE.md, ROADMAP.md, DATABASE_DESIGN.md, API_DOCUMENTATION.md and DEVELOPMENT_GUIDE.md. The Go runbook is intended as a useful public learning and reference document for other developers, not merely a personal notebook.

## 3.21 Product Roadmap

The roadmap organises the work into four versions, summarised in Figure 3.7 and Table 3.14. An important correction to early notes should be recorded: build history was part of the early DevOps Analytics idea, but the refined version 1 scope concentrates on GitHub integration and basic GitHub activity. CI/CD build and deployment analytics were moved into the later delivery-analytics phase.

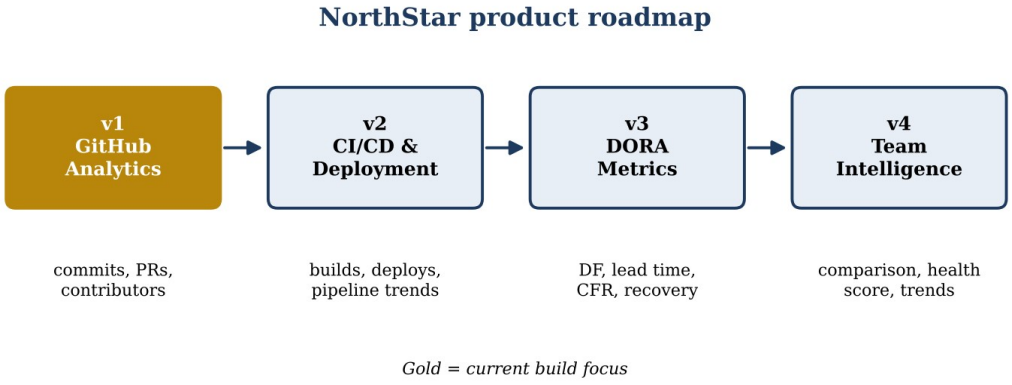



**Figure 3.7: NorthStar product roadmap**

**Table 3.14: Roadmap by version**

| Version | Theme | Planned capabilities |
|---|---|---|
| v1 | Engineering activity: GitHub analytics | GitHub integration; commits; pull requests; contributors; repository insights; basic dashboard. |
| v2 | Delivery activity: CI/CD and deployments | Build history; build success and failure; build duration; deployment history; pipeline trends. |
| v3 | DORA intelligence | Deployment frequency; lead time for changes; change failure rate; failed-deployment recovery time. |
| v4 | Team intelligence | Team comparison; engineering health score; sprint analytics; executive dashboard; historical trends; cross-project insights. |

### 3.21.1 What Was Deliberately Excluded from Version 1

A significant architectural decision was to avoid turning the first release into a very large platform. The following were deliberately excluded from version 1: Jenkins, Jira and Kubernetes integrations; the full DORA implementation; AI-based recommendations; complex event infrastructure; multiple microservices; and a sophisticated authentication system. They remain possible future capabilities and not version 1 requirements.

### 3.21.2 Architectural Traps to Avoid

1. Do not start with microservices; one Go application with clean internal modules is sufficient for version 1.
2. Do not introduce Kubernetes to make the project look advanced; first make NorthStar work reliably under Docker.
3. Do not introduce Kafka, NATS or RabbitMQ before a real event-processing requirement exists.
4. Do not add AI merely to put AI in the description; any future insights must answer meaningful questions using reliable data.
5. Do not confuse dashboard development with product development; the dashboard is the presentation layer, and the core product is the pipeline, integrations, processing and analytics.
6. Do not over-engineer version 1; a small, complete and tested product is more valuable than a large architecture that only starts successfully.

# Chapter 4
# Experimentation & Results Discussion

## 4.1 Introduction and Methodology

This chapter reports what has actually been done and verified, what has been designed but is still to be verified, and how the remaining work will be evaluated. It is deliberately candid. NorthStar is a systems-engineering project whose value will ultimately be judged by whether the end-to-end workflow runs correctly. At the time of this report, the project is at the foundation stage and the GitHub integration is the current development milestone. Presenting benchmark figures for a pipeline that is not yet complete would be misleading, so the chapter distinguishes three categories of evidence:

- **Verified results:** outcomes that were observed by running the application or inspecting the repository, namely the application foundation, configuration, logging, container setup and the secret-leak remediation.
- **Designed but pending verification:** components whose design is complete but whose implementation status must be confirmed against the repository, namely the GitHub client and its helper methods.
- **Planned evaluation:** the test strategy, acceptance criteria and metric-validation approach that will be applied as each component is completed.

The project follows an incremental sprint methodology influenced by agile practice [27]. Sprint 1 established the foundation. Sprint 2 began the GitHub integration. Each sprint is scoped to produce a working, demonstrable increment and ends with a verification step in which the application is run and its output inspected.

## 4.2 Experimental Environment

**Table 4.1: Development and experimentation environment**

| Item | Description |
|---|---|
| Development machine | Personal laptop with a six-core processor exposing twelve logical CPUs (as discussed in the Go scheduler study) and a discrete GPU; used only as a development workstation. |
| Operating environment | Docker Engine / Docker Desktop with Docker Compose; code editor and Git. |
| Language and runtime | Go toolchain (module-based project). |
| Database | PostgreSQL 17 container with a named volume. |
| Source hosting | GitHub: public repository Raunak08-code/NorthStar, default branch main. |

| Item | Description |
|---|---|
| External data source | GitHub REST API accessed with a personal access token supplied through the environment. |

## 4.3 Verification 1: Application Foundation

The first verification concerns the Go application foundation created in Sprint 1: module initialisation, the main.go entry point, configuration loading and the logger. The application was run, and the output below was recorded as the last confirmed run.

```
NorthStar v1
Insights for Every Development Journey.
========================================
✓ Logger Initialized
  Configuration Loaded
  Server Port : 8080
  NorthStar Started
```

*Listing 4.1: Output of the last confirmed application run*

**Result.** The output confirms that the application compiles and runs; that the logger is initialised before other components, which allows all later steps to log; that configuration is loaded and the server port is read from configuration; and that the startup sequence completes. This validates the foundation described in the composition-root design: main.go loads configuration, initialises the logger and starts the application.

**Interpretation and limits.** The startup output does not demonstrate that the GitHub collector, database integration, REST API or dashboard are working. Those components are not part of this verification, and nothing in this report should be read as claiming otherwise. The result establishes the base on which they are built.

## 4.4 Verification 2: Configuration Management

Configuration was implemented in a dedicated package, with the Config structure described in Chapter 3. The verification checked three behaviours: that values are read from environment variables, that a default is used for the port when none is supplied, and that the application reports a clear error when a required value is absent. The intended behaviours and their status are summarised in Table 4.2.

**Table 4.2: Configuration behaviours and verification status**

| Behaviour | Expected result | Status |
|---|---|---|
| Read PORT from environment | Server port taken from configuration (8080 observed). | Verified |

| Behaviour | Expected result | Status |
|---|---|---|
| Load GitHub token from environment | Token available to the GitHub client without being printed. | Design complete; confirm in repository |
| Parse polling interval | Duration value parsed from text such as 5m. | Design complete; confirm in repository |
| Fail fast on missing required value | Descriptive error at startup, not later at first use. | Design complete; confirm in repository |
| Keep .env out of Git | Real .env ignored; .env.example committed. | Verified (see 4.6) |

## 4.5 Verification 3: Container Foundation

The container environment was built in Sprint 1. A multi-stage Dockerfile compiles the Go application; Docker Compose runs the application and a PostgreSQL 17 container; a named volume stores database data; and configuration is supplied through environment variables. Two behaviours were studied and verified conceptually and practically.

### 4.5.1 Database Host Name inside Docker Compose

A database URL used from the host machine can legitimately use localhost, because the PostgreSQL container publishes its port to the host. The same URL used by the NorthStar container fails, because inside that container localhost refers to the container itself. The correct URL for the application container uses the Compose service name:

```
# from the host machine (e.g. a local psql client)
postgres://postgres:postgres@localhost:5432/northstar?sslmode=disable

# from the NorthStar container inside the Compose network
postgres://user:password@postgres:5432/northstar?sslmode=disable
```

*Listing 4.2: Database URLs for host and container contexts*

**Result.** The correct rule is that the host portion is the Compose service name for container-to-container traffic. This resolved a class of connection errors and is now recorded in the project documentation.

### 4.5.2 Volume Persistence

The behaviour of Compose commands with respect to named volumes was tested and is summarised in Table 4.3.

**Table 4.3: Effect of Compose shutdown commands on data**

| Command | Containers and network | Named volume (database data) |
|---|---|---|
| docker compose down | Removed | Preserved |

| Command | Containers and network | Named volume (database data) |
|---|---|---|
| docker compose down -v | Removed | Removed (data lost) |
| docker compose stop / start | Stopped / restarted | Preserved |

**Result.** Persisted PostgreSQL data survives routine restarts and docker compose down, and is deleted only when the -v flag is used. The practical guidance adopted is to use down -v deliberately and only when a clean database is wanted.

## 4.6 Verification 4: Secret-Leak Incident and Remediation

A significant practical event during Sprint 1 was the accidental commit of a GitHub Personal Access Token in a .env file. GitHub push protection blocked the push and identified the exposed token in an earlier commit. Although it is an incident and not a planned experiment, it produced results of real value and is reported here as a case study in secure engineering.

**Table 4.4: Timeline and remediation of the token exposure**

| Step | Event / action | Outcome and lesson |
|---|---|---|
| 1 | A real token was written to .env and the file was committed. | The file was tracked by Git; .gitignore does not affect files that are already tracked. |
| 2 | git push was attempted. | GitHub push protection blocked the push and identified the secret in a previous commit. |
| 3 | The secret was removed from the working tree in a later commit. | This is insufficient: the secret remains in earlier history. |
| 4 | The exposed token was revoked in GitHub and a replacement created. | A leaked credential must be considered compromised and must be revoked and replaced. |
| 5 | The repository history was rebuilt without the secret. | The history was cleaned and the repository re-published. The public repository currently shows a short, clean history. |
| 6 | .env was added to .gitignore and .env.example committed with placeholders. | Configuration is now safe to publish. |

**Results and rules adopted.** (1) .gitignore must be in place before the first commit of any secret-bearing file. (2) Deleting a secret in a later commit does not remove it from history. (3) A leaked credential must be revoked and replaced immediately, regardless of how quickly the push was blocked. (4) History must be cleaned when necessary. (5) .env.example holds placeholders only. (6) Tokens are scoped minimally. (7) Platform protections such as push protection are valuable defence in depth but are not a substitute for correct practice [28].

## 4.7 Verification 5: Repository Audit

As part of preparing this report, the public GitHub repository was reviewed to establish the actual state of the project and to prevent overstatement. The repository https://github.com/Raunak08-code/NorthStar is public, uses main as its default branch, and shows two commits. Its top level contains the items listed in Table 4.5.

**Table 4.5: Top-level contents of the repository at the time of review**

| Item | Observation |
|---|---|
| Documents/ | Documentation directory. |
| internal/ | Directory for internal application packages (configuration, logger and, prospectively, integrations). |
| main.go | Application entry point. |
| go.mod, go.sum | Go module definition and dependency checksums. |
| Dockerfile, .dockerignore | Container build definition and exclusions. |
| docker-compose.yml | Local orchestration of the application and PostgreSQL. |
| .env.example, .gitignore | Configuration template and ignore rules. |
| README.md | Repository description: the name, tagline and a one-line description of the platform; to be expanded. |

**Result.** The repository confirms the foundation-stage status: an entry point, module files, Docker and Compose files, a safe configuration template and documentation and internal-package directories. The short history of two commits is consistent with the rebuild of the history after the token incident. No pull requests, issues, tags or releases exist yet. The README is minimal and should be expanded using the structure proposed in the planning documents once version 1 is functional.

## 4.8 Progress Summary

**Table 4.6: Development progress at the time of submission**

| Workstream | Status | Evidence |
|---|---|---|
| Go module and main.go | Completed | Verified run (Listing 4.1); repository files. |
| Configuration package | Completed (foundation) | Port loading verified; repository. |
| Logger package | Completed (foundation) | Verified run output. |
| Dockerfile and Compose | Initial setup completed | Files in repository. |
| PostgreSQL 17 container | Configured | Compose file; networking and volume behaviour studied. |

| Workstream | Status | Evidence |
|---|---|---|
| Secret hygiene | Completed | Incident remediated; .env.example in repository. |
| GitHub HTTP client | In progress; design completed | Design in Listing 3.2; implementation to be confirmed. |
| GitHub models and operations | Planned | — |
| Collector, database schema, analytics | Planned | — |
| REST API, scheduler | Planned | — |
| Frontend dashboard | Planned | — |
| Automated tests and CI pipeline | Planned | — |

The progress table shows that the foundation work, which is necessary for everything else, is complete, and that the principal remaining effort is the vertical slice from GitHub through storage and analytics to the API and dashboard. The original target of completing NorthStar by August of the planning year was not met; the schedule slipped, and it was decided not to compress months of engineering into a few days but to complete the roadmap in its natural order.

## 4.9 Evaluation Plan for Version 1

The remaining components will be evaluated at several levels. The plan below is part of the framework, since the quality of an analytics platform depends on the correctness of its numbers.

**Table 4.7: Test levels and planned validation**

| Test level | Planned validation |
|---|---|
| Unit tests | Configuration parsing; model mapping from JSON; metric calculation functions with known inputs and expected outputs; helper functions. |
| HTTP client tests | Using an httptest mock server: request headers, bearer authentication, User-Agent, timeout behaviour, handling of 4xx and 5xx responses, malformed JSON, pagination across several pages, rate-limit headers. |
| Integration tests | Database reads and writes in a disposable PostgreSQL container; idempotent upserts; collection against controlled test data. |
| API tests | Endpoint status codes, JSON structure, input validation, pagination bounds and error responses. |
| End-to-end tests | Collect a sample repository, persist it, calculate metrics and retrieve them through the API. |
| Manual verification | Run the application and containers, inspect logs, confirm that no secret appears in output, and check dashboard values against GitHub’s own UI. |

### 4.9.1 Acceptance Criteria for GitHub Version 1

1. A valid GitHub token can be used without being exposed in logs or source control.
2. The application can retrieve a selected repository's metadata.
3. Commit and pull-request records are mapped into internal models.
4. Repeated collection does not create duplicate records for the same source objects.
5. Basic metrics can be retrieved through documented API endpoints and match independently computed values for a test repository.
6. Failures are logged with enough context to support debugging, and no log line contains a secret.

### 4.9.2 Validating Metric Correctness

The most important evaluation for an analytics platform is cross-validation of its numbers. For a small test repository, the expected counts of commits and pull requests will be obtained independently, for example from the GitHub web interface or from the git log command, and compared with NorthStar's output for the same period. Discrepancies reveal pagination errors, time-zone errors and filtering mistakes. Metric functions will additionally be tested with hand-computed fixtures, as illustrated in the next section.

## 4.10 Worked Example of Metric Computation (Illustrative Data)

> *The numbers in this section are hypothetical sample data, invented only to demonstrate how NorthStar's metric definitions behave. They are not measurements from a real repository.*

Consider a sample repository with eight pull requests merged during a month. The time from creation to merge, in hours, was 4, 26, 8, 52, 3, 15, 30 and 6. The sum is 144 hours, so the mean time to merge is 144 / 8 = 18.0 hours. Sorting the values gives 3, 4, 6, 8, 15, 26, 30 and 52. The median is the average of the fourth and fifth values, (8 + 15) / 2 = 11.5 hours. The mean is 56 per cent higher than the median because of the single 52-hour pull request. This illustrates why the definition in Table 3.11 states that the mean is skewed by outliers and why the dashboard will show median and percentiles alongside the mean.

Suppose weekly commit counts for the month were 14, 22, 9 and 19. The total is 64 commits, an average of 16 per week, with the third week noticeably lower. A raw number such as 64 says nothing about quality; it only describes activity, which is why the report and the planned dashboard consistently label such measures as activity metrics.

For a hypothetical delivery pipeline over a two-week period, suppose ten production deployments occurred, two of which failed, with recovery times of 40 and 100 minutes, and five

changes had lead times of 6, 10, 20, 30 and 50 hours. Table 4.8 applies the formulas of Table 3.12 and compares them with the example goals recorded in the project notes.

**Table 4.8: Worked DORA example with hypothetical data**

| Metric | Calculation | Result | Against example goal |
|---|---|---|---|
| Deployment frequency | 10 deployments / 2 weeks | 5 per week | Below goal of more than 20 per week |
| Lead time for changes (median) | Median of 6, 10, 20, 30, 50 h | 20 hours | Meets goal of less than 24 h |
| Lead time for changes (mean) | 116 h / 5 | 23.2 hours | Meets goal, but close |
| Change failure rate | 2 failed / 10 deployments | 20 per cent | No default target |
| Failed-deployment recovery time (mean) | (40 + 100) min / 2 | 70 minutes | No default target |
| Build success rate (v2 example) | 47 successful / 50 builds | 94 per cent | Below goal of more than 95 per cent |

The example shows how the goals pillar will work: the platform compares each metric with the target defined by the manager and reports whether it is met, while the intelligence layer can add context. It also shows why definitions matter: the choice of median or mean changes the apparent lead time and, in a different data set, could change whether the goal is met.

## 4.11 Discussion

### 4.11.1 What the Results Show

The verified results establish that the foundation of NorthStar is sound: a Go application that starts, loads configuration and logs; a reproducible container environment; a safe configuration approach; and a clean public repository. These are modest but necessary achievements, and they were obtained while following the principle that complexity should be introduced only when needed.

### 4.11.2 Learning Outcomes

Because NorthStar was also a practical Go learning project, a body of learning emerged that is worth preserving and that also informs the design.

- **Compiled language.** Go source is compiled by the Go compiler into a native executable that runs directly on the operating system and CPU, with no traditional interpreter. Compilation is not the only reason programs are fast: network latency, I/O, memory allocation and application design often matter just as much.

- **Process, thread and goroutine.** A process is a running program with its own resources and address space. An operating-system thread is a unit of execution scheduled by the OS. A goroutine is a lightweight unit of execution managed by the Go runtime; it is not an OS thread, and many goroutines can be multiplexed onto fewer threads.
- **Concurrency versus parallelism.** Concurrency means multiple tasks make progress during overlapping periods; parallelism means tasks execute at the same instant on different resources. A program can be concurrent without everything running simultaneously [26].
- **The G-M-P scheduler.** Goroutines (G) wait in run queues; processors (P) are logical scheduling resources; machines (M) are OS threads. A blocked goroutine is parked while its thread runs another goroutine, goroutines are not permanently bound to a thread, and work stealing balances load. On a machine with six physical cores and twelve logical CPUs, many more goroutines than CPUs can exist, but the amount of work executing simultaneously is bounded by the available execution resources and the scheduler configuration.
- **The wider runtime.** The scheduler is only part of the Go runtime, which also performs garbage collection, memory allocation, stack management, timers and network polling.
- **Composition root.** main.go should explain how NorthStar starts, not how it works, which keeps business logic testable and the entry point readable.
- **Practical operations.** Docker networking (service names instead of localhost), volume semantics, and the permanent nature of secrets in Git history.

### 4.11.3 Design Decisions Validated

Several design decisions have already proved their value. Choosing a single modular application avoided the operational overhead of multiple services at a stage where there is only one integration. Keeping the constructor free of network calls and placing the connectivity check in main.go leads to a more testable design. Excluding Kafka, Kubernetes and AI from version 1 kept attention on the core data pipeline. Writing precise metric definitions early surfaced ambiguities, such as the three possible meanings of contributor activity, before any code depended on them.

### 4.11.4 Threats to Validity and Limitations

- **Scope of evidence.** Verified results cover the foundation only. No claims are made about collector throughput, analytics accuracy or dashboard usability, since these components are not complete.
- **Single data source.** Version 1 relies on GitHub alone, so cross-tool lifecycle metrics are not yet possible.

- **Data quality dependence.** Metrics depend on the completeness and consistency of the source data. For example, squash merges, rebases and bots can distort commit-based measures.
- **API constraints.** GitHub rate limits, pagination limits and API changes can affect collection and require ongoing maintenance.
- **Metric interpretation.** Even correct metrics can be misused. Dashboards must carry definitions and limitations, and the platform should not be used to rank individuals [3], [19], [20].
- **Single-developer context.** Design reviews, load testing and usability evaluation with real teams have not yet been performed.

# Chapter 5
# Conclusion and Future Work

## 5.1 Conclusion

This report presented NorthStar, an Engineering Intelligence Platform whose purpose is to help engineering organisations understand how software is built, tested, deployed and improved. The project began from a simple observation: the information needed to answer “how are we doing?” is scattered across source-control platforms, CI/CD systems, issue trackers, deployment platforms and monitoring tools, and every person on a team therefore gives a different answer. NorthStar proposes to collect that information, connect it into a coherent journey from commit to recovery, compute well-defined metrics from it and present the results to the people who need them.

The report reviewed the research that informs the design, namely the DORA four key metrics, the SPACE framework and the cautions of Goodhart’s law, and the engineering practice of continuous delivery and site reliability engineering. It positioned NorthStar relative to existing approaches and identified the opportunity for a transparent, modular, incrementally built implementation.

The proposed framework combines six pillars (source integration, analytics engine, intelligence layer, dashboards, reports and goals) with a pragmatic architecture. The long-term architecture is event-oriented, but version 1 is intentionally a modular Go application with an integration layer, collector, analytics, scheduler, database and REST API, backed by PostgreSQL and packaged with Docker. The report documented detailed design for the GitHub client, the collection strategy, the data model, metric definitions with their limitations, the API, configuration and security.

In terms of implementation, the foundation of the project is in place and has been verified: the Go application starts, loads configuration and logs; the Docker and PostgreSQL environment is configured; the public repository is established; and a real secret-leak incident was handled and converted into lasting engineering rules. The GitHub integration is the current milestone, and the collector, persistence, analytics, API, scheduler, dashboard and automated test suite remain to be built. The report has been careful to distinguish clearly between the implemented foundation and the designed-but-unbuilt remainder.

Beyond the artefact itself, the project has been a substantive learning exercise in the engineering of infrastructure-oriented software: Go’s compilation model, goroutines and the G-M-P scheduler, HTTP client design, configuration management, container networking and

volumes, repository hygiene and secure handling of credentials. The most valuable general lesson is architectural restraint: a small, complete and tested product is worth more than a grand architecture that merely starts.

## 5.2 Future Work

### 5.2.1 Completing Version 1

The immediate priority is to complete the vertical slice that defines version 1. The recommended sequence, which preserves the architecture while making the order explicit, is shown in Table 5.1.

**Table 5.1: Implementation roadmap from the current position**

| Stage | Work | Deliverable |
|---|---|---|
| 0 | Re-entry and repository audit | Know the actual current state of the code. |
| 1 | Complete the GitHub client | Authenticated, reusable API client with tests. |
| 2 | GitHub models | Go structures for API responses. |
| 3 | Repository operations | Retrieve repository information. |
| 4 | Commit operations | Retrieve and process commits with pagination. |
| 5 | Pull-request operations | Retrieve pull requests and their states. |
| 6 | Contributor operations | Retrieve contributor information. |
| 7 | Collector | Coordinate data collection across repositories. |
| 8 | Database | Schema, migrations and idempotent persistence. |
| 9 | Analytics | Calculate initial engineering metrics. |
| 10 | REST API | Expose stored information and metrics. |
| 11 | Scheduler | Periodically refresh data. |
| 12 | Testing | Unit and integration tests; CI pipeline. |
| 13 | Frontend | Basic NorthStar dashboard. |
| 14 | Documentation and release | README, API documentation, version 1 demonstration and release. |

### 5.2.2 Subsequent Versions

- **Version 2, CI/CD and deployment analytics:** workflow runs, build duration, success and failure, deployment history and pipeline trends, starting with GitHub Actions and later Jenkins.
- **Version 3, DORA metrics:** deployment frequency, lead time for changes, change failure rate and failed-deployment recovery time, including a reliable association between commits and deployments, and incident or rollback signals.

- **Version 4, team intelligence:** team comparison, engineering health score, sprint analytics, bottleneck detection, executive dashboard and historical trends, with attention to responsible presentation.

### 5.2.3 Platform Enhancements

- Additional integrations: GitLab, Jira, Jenkins and Kubernetes, each as a package under internal/integrations.
- Webhook-based collection and, when justified by real throughput requirements, asynchronous event processing with a message broker such as NATS or RabbitMQ.
- Caching with Redis for expensive dashboard queries.
- Operational observability for NorthStar itself using Prometheus and Grafana.
- Authentication and role-based access control, multi-team and multi-organisation support.
- Orchestrated deployment on Kubernetes once the application is reliable under Docker.
- Configurable goals and alerts, scheduled report generation and export in common formats.
- A Python-based analytics component and, only where supported by reliable and traceable data, AI-assisted summaries or recommendations.

### 5.2.4 Engineering and Community Practice

The project should also grow in maturity as an open-source repository: an expanded README that clearly distinguishes current from planned features, an architecture diagram, setup and configuration instructions, API documentation, screenshots and demonstrations, a contributing guide and licence. The Go runbook, a document describing Go concepts encountered in the project, is intended to become a public learning resource.

## 5.3 Final Remarks

NorthStar demonstrates how a clear problem statement, disciplined scoping and honest reporting of progress can turn a broad idea into an executable plan. Its long-term value will depend on accurate data, clear metric definitions and insights that help teams improve their delivery, not merely measure it. The foundation laid so far, and the design documented in this report, provide a credible path to that goal.

# References

[1] N. Forsgren, J. Humble and G. Kim, Accelerate: The Science of Lean Software and DevOps. Portland, OR, USA: IT Revolution Press, 2018.

[2] DORA (DevOps Research and Assessment), Google Cloud, “State of DevOps Reports and software delivery performance resources.” [Online]. Available: https://dora.dev/

[3] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck and J. Butler, “The SPACE of Developer Productivity,” ACM Queue, vol. 19, no. 1, 2021.

[4] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.

[5] G. Kim, J. Humble, P. Debois and J. Willis, The DevOps Handbook. Portland, OR, USA: IT Revolution Press, 2016.

[6] B. Beyer, C. Jones, J. Petoff and N. R. Murphy (Eds.), Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA, USA: O’Reilly Media, 2016.

[7] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[8] “Go Documentation,” The Go Programming Language. [Online]. Available: https://go.dev/doc/

[9] “Package net/http,” Go Packages. [Online]. Available: https://pkg.go.dev/net/http

[10] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[11] “GitHub REST API Documentation,” GitHub Docs. [Online]. Available: https://docs.github.com/en/rest

[12] “PostgreSQL Documentation,” The PostgreSQL Global Development Group. [Online]. Available: https://www.postgresql.org/docs/

[13] “Docker Documentation,” Docker Inc. [Online]. Available: https://docs.docker.com/

[14] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” Linux Journal, no. 239, 2014.

[15] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O’Reilly Media, 2017.

[16] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2021.

[17] M. Fowler, “MonolithFirst,” martinfowler.com, 2015. [Online]. Available: https://martinfowler.com/bliki/MonolithFirst.html

[18] G. Hohpe and B. Woolf, Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston, MA, USA: Addison-Wesley, 2003.

[19] M. Strathern, “‘Improving ratings’: audit in the British University system,” European Review, vol. 5, no. 3, pp. 305–321, 1997.

[20] J. Z. Muller, The Tyranny of Metrics. Princeton, NJ, USA: Princeton University Press, 2018.

[21] M. Skelton and M. Pais, Team Topologies: Organizing Business and Technology Teams for Fast Flow. Portland, OR, USA: IT Revolution Press, 2019.

[22] “Prometheus Documentation.” [Online]. Available: https://prometheus.io/docs/

[23] “Grafana Documentation,” Grafana Labs. [Online]. Available: https://grafana.com/docs/

[24] “Kubernetes Documentation,” The Kubernetes Authors. [Online]. Available: https://kubernetes.io/docs/

[25] “Git Documentation.” [Online]. Available: https://git-scm.com/doc

[26] R. Pike, “Concurrency is not parallelism,” talk at Heroku Waza, 2012.

[27] K. Beck et al., “Manifesto for Agile Software Development,” 2001. [Online]. Available: https://agilemanifesto.org/

[28] “About secret scanning and push protection,” GitHub Docs. [Online]. Available: https://docs.github.com/en/code-security/secret-scanning

[29] “Jenkins User Documentation.” [Online]. Available: https://www.jenkins.io/doc/

[30] “GitHub Actions Documentation,” GitHub Docs. [Online]. Available: https://docs.github.com/en/actions

[31] “RabbitMQ Documentation,” Broadcom. [Online]. Available: https://www.rabbitmq.com/docs

[32] “Redis Documentation.” [Online]. Available: https://redis.io/docs/

[33] “React Documentation.” [Online]. Available: https://react.dev/

[34] A. Wiggins, “The Twelve-Factor App,” 2011. [Online]. Available: https://12factor.net/

# Appendix A: Glossary

**Table A.1: Glossary of terms**

| Term | Definition |
|---|---|
| API | Application Programming Interface; a defined way for software components to communicate. |
| CI/CD | Continuous Integration / Continuous Delivery or Deployment; automated build, test and release practices. |
| Collector | Component responsible for retrieving data from an external source. |
| Commit | A recorded set of changes in a version-control system, identified by a hash (SHA). |
| Composition root | The place in an application, usually main, where components are created and wired together. |
| DORA | DevOps Research and Assessment; research programme behind the four key delivery metrics. |
| Event-driven architecture | An approach in which components communicate or react to events representing changes or actions. |
| Goroutine | A lightweight execution unit managed by the Go runtime. |
| G-M-P | Goroutine, Machine (OS thread), Processor (logical scheduling resource): the Go scheduler model. |
| HTTP | Hypertext Transfer Protocol. |
| Idempotency | The property that performing an operation more than once has the same effect as performing it once. |
| Integration | A module that communicates with an external engineering platform. |
| JSON | JavaScript Object Notation; a text data interchange format. |
| Lead time for changes | Time from a change being committed to it running in production. |
| Modular monolith | A single deployable application organised into clearly separated internal modules. |
| Normalisation | Converting source-specific data into a consistent internal representation. |
| Pull request (PR) | A proposal to merge code changes into a repository branch. |
| REST | Representational State Transfer; an HTTP-based architectural style for APIs. |
| SRE | Site Reliability Engineering. |
| Upsert | A database operation that inserts a row or updates it if it already exists. |
| Webhook | An HTTP callback in which one system notifies another when an event occurs. |

# Appendix B: Abbreviations

**Table B.1: Abbreviations used in this report**

| Abbreviation | Meaning |
|---|---|
| API | Application Programming Interface |
| CFR | Change Failure Rate |
| CI/CD | Continuous Integration / Continuous Delivery (or Deployment) |
| DF | Deployment Frequency |
| DORA | DevOps Research and Assessment |
| HTTP | Hypertext Transfer Protocol |
| JSON | JavaScript Object Notation |
| MTTR | Mean Time To Recovery / Restore |
| PR | Pull Request |
| REST | Representational State Transfer |
| SHA | Secure Hash Algorithm (commit identifier) |
| SLA | Service-Level Agreement |
| SQL | Structured Query Language |
| SRE | Site Reliability Engineering |
| UI | User Interface |
| VCS | Version Control System |

# Appendix C: Illustrative API Response

The following listing is an illustrative example of the response format planned for a metric endpoint. It is a design example, not output from a running system. Note that the definition is returned with the value so that dashboards can display it beside the number.

```
GET /api/v1/repositories/42/metrics?from=2026-08-01&to=2026-08-31

{
  "repository_id": 42,
  "period": { "from": "2026-08-01", "to": "2026-08-31" },
  "metrics": [
    {
      "name": "pull_requests_merged",
      "value": 8,
      "definition": "Pull requests with merged_at inside the period."
    },
    {
      "name": "time_to_merge_median_hours",
      "value": 11.5,
      "definition": "Median of merged_at - created_at for merged PRs.",
      "limitation": "Excludes PRs that were closed without merging."
    }
  ]
}
```

*Listing C.1: Illustrative metric endpoint response (hypothetical values)*

# Appendix D: Project Planning Notes Summary

The original handwritten planning notes for NorthStar recorded the key ideas that evolved into this report. They are summarised here for completeness.

- **Audience:** NorthStar is “not for everyone”: team leads, project managers, engineering managers, DevOps engineers and CTOs.
- **Insight data:** deployment rate, rollbacks, versions, failures, success rate and commits.
- **Capabilities by version:** developer productivity, deployment frequency, lead time and DORA metrics, build success and failure trends, team performance, engineering velocity, bottlenecks and GitHub contribution patterns.
- **Problem it solves:** when a manager asks “how are we doing?”, everybody in the team has a different answer; NorthStar provides one answer by connecting those data points.
- **Core idea:** every engineering activity leaves a footprint; NorthStar connects every step from commit through recovery.
- **Vision statement:** NorthStar helps engineering organisations understand how software is built, tested, deployed and improved by transforming deployment data into meaningful insights.
- **Workflow:** user connects GitHub (dashboard, connect repository, GitHub authentication, token); the collector runs every five minutes to get commits, pull requests, actions and releases and stores raw data (collection only, no calculation); the analytics engine calculates lead time, deployment frequency and build success and stores the analytics; the dashboard requests already-calculated metrics.
- **Four major services:** collector (collect data), processing (transform data), analytics (compute metrics) and API (serve dashboard).